



Focus: A Streaming Concentration Architecture for Efficient Vision-Language Models

Chiyue Wei^{†*}, Cong Guo^{†*§}, Junyao Zhang[†], Haoxuan Shan[†], Yifan Xu[†], Ziyue Zhang[†],
Yudong Liu[†], Qinsi Wang[†], Changchun Zhou[†], Hai “Helen” Li[†], Yiran Chen[†]

[†]Duke University ^{*}Equal contribution [§]Corresponding author
{chiyue.wei, cong.guo, hai.li, yiran.chen}@duke.edu

Abstract—Vision-Language Models (VLMs) have demonstrated strong performance on tasks such as video captioning and visual question answering. However, their growing scale and video-level inputs lead to significant computational and memory overhead, posing challenges for real-time deployment on hardware accelerators. While prior work attempts to reduce redundancy via token pruning or merging, these methods typically operate at coarse granularity and incur high runtime overhead due to global token-level operations.

In this study, we propose *Focus*, a *Streaming Concentration Architecture* that efficiently accelerates VLM inference through progressive, fine-grained redundancy elimination. *Focus* introduces a multilevel concentration paradigm that hierarchically compresses vision-language inputs at three levels: (1) semantic-guided token pruning based on textual prompts, (2) spatial-temporal block-level concentration using localized comparisons, and (3) vector-level redundancy removal via motion-aware matching. All concentration steps are tightly co-designed with the architecture to support streaming-friendly, on-chip execution. *Focus* leverages GEMM tiling, convolution-style layout, and cross-modal attention to minimize off-chip access while enabling high throughput. Implemented as a modular unit within a systolic-array accelerator, *Focus* achieves 2.4 $\times$ speedup and 3.3 $\times$ reduction in energy, significantly outperforming state-of-the-art accelerator in both performance and energy efficiency. Full-stack implementation of *Focus* is open-sourced at <https://github.com/dubeyfor3/Focus>.

I. INTRODUCTION

Vision-Language Models (VLMs) [35], [41] have emerged as a cornerstone of multimodal AI, enabling joint reasoning over visual and textual data. By integrating advances from computer vision and natural language processing, VLMs excel at tasks such as video captioning [67], [74], visual question answering [13], [55], and cross-modal retrieval [36]. Following a similar trajectory to Large Language Models (LLMs) [6], [15], modern VLMs have rapidly scaled in size and data, resulting in notable accuracy gains. However, this scaling significantly increases compute and memory demands, posing challenges for deployment, especially on edge devices [53].

Fortunately, video-based inputs offer a key opportunity: high visual redundancy [7], [28], [52], [62], [69]. As shown in Fig. 1(a), adjacent frames often share similar backgrounds and foreground objects. Since VLMs tokenize each frame independently [35], [74], many tokens across or within frames are redundant. This has motivated techniques such as token pruning [50], [62] and token merging [4] to reduce computation. However, most prior work focuses on algorithmic strategies without considering hardware alignment. For instance, Token

Merging [4] introduces a ToMe module that increases runtime by up to 36.8% [70].

Recent designs such as AdapTiV [70] and CMC [56] address these inefficiencies at the hardware level. AdapTiV implements a simplified ToMe module in hardware, while CMC leverages video-codec-inspired compression (e.g., H.264 [65]) via an external codec block. However, both approaches largely translate existing algorithms without embracing full hardware-algorithm co-design. First, both targeted for Vision Transformers (ViTs) [17], focus only on visual redundancy and overlook the cross-modal nature of VLMs. CMC’s codec ignores language inputs, and AdapTiV only supports static images, missing video-language interactions. Second, both operate at global token-level granularity, which is inefficient for both algorithm and hardware due to high overhead and poor locality. To enable efficient VLM deployment, a more holistic co-design approach is needed, one that leverages cross-modal redundancy while aligning with hardware-friendly processing granularity.

In this study, we propose a novel architecture, *Focus*, to accelerate VLM inference by performing *streaming concentration*, a multilevel compression technique that removes visual and cross-modal redundancy in a streaming-friendly, on-chip processing fashion.

From the **algorithmic perspective**, *Focus* performs redundancy concentration at three levels of granularity. First, it leverages semantic understanding to retain only visual regions relevant to the textual prompt. Prior work [4], [49], [56], [70] relies on static metrics like token magnitude, which fail to capture prompt-conditioned semantics in VLMs. As shown in Fig. 1(a), attention may shift from a foreground object (e.g., a dog) to a background element (e.g., a flower), depending on the question (see details in Sec. III-A). To address this, *Focus* introduces a prompt-aware importance analyzer that dynamically prunes visual tokens based on cross-modal attention, improving both accuracy and efficiency.

Second, as illustrated in Fig. 1(b), *Focus* groups retained tokens into spatiotemporal blocks, using the last token (e.g., token *h*) as the key for localized similarity comparisons. The key token is compared with others in its block. This is applied across the video, treating each token in turn as a key. This technique resembles a 3D convolutional sweep that progressively concentrates similarity through localized matching. By operating within small spatial-temporal windows, *Focus* avoids

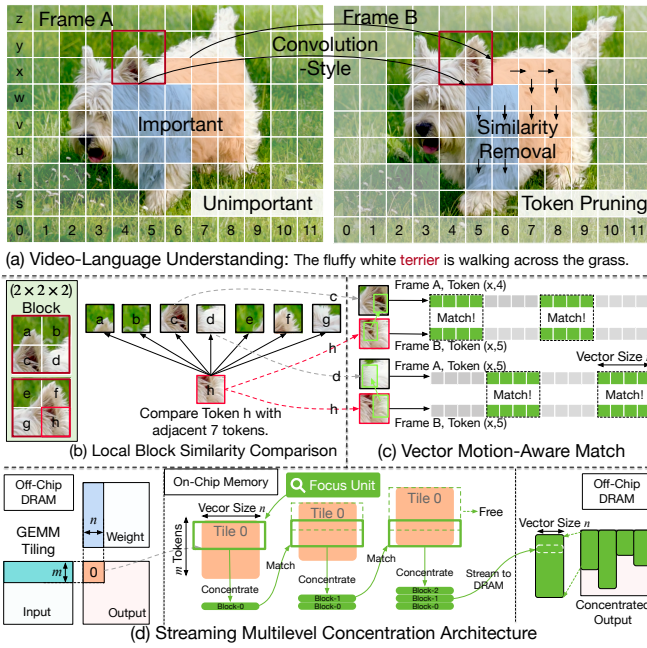


Fig. 1. Overview of the streaming multilevel concentration architecture.

global comparisons, making the process compute-efficient and highly streamable.

Third, *Focus* explores redundancy at the vector level. Due to video motion, a token may align with multiple shifted tokens in adjacent frames. As shown in Fig. 1(c), token *h* may share features with parts of tokens *c* and *d*. Relying on a single best-matched token may lose information. Instead, *Focus* performs vector-wise comparisons, allowing each vector to match multiple candidates and capture richer sub-token similarity. By integrating these three levels, *Focus* achieves up to **83%** (**80%** on average) computational sparsity through multilevel concentration, significantly outperforming CMC and AdapTiV, which typically reach only 40–50%, under similar accuracy.

From the **architectural perspective**, *Focus* is designed to efficiently support multilevel concentration through tight alignment with General Matrix Multiplication (GEMM) tiling. As shown in Fig. 1(d), its vector- and block-level operations naturally align with the tiling strategies widely adopted in systolic-array-based accelerators such as TPUs [31] and GPUs [11]. GEMM tiling addresses on-chip memory constraints by dividing matrices into small, independently processed tiles. Each tile is handled by the PE array in isolation, enabling efficient, in-place **vector-level** similarity detection and compression. By eliminating redundancy locally within each tile, *Focus* minimizes data movement and reduces both compute cost and DRAM traffic. In contrast to global token-wise methods that rely on costly off-chip access, *Focus* achieves fine-grained, on-chip processing in a hardware-efficient manner.

At the **block level**, *Focus* draws inspiration from CNN accelerators [8], [18], using a sliding window to stream and process output tokens directly from the compute core (e.g., systolic array), maximizing locality and sustaining high

throughput. To handle the non-contiguous nature of VLM tokens within a block, we adopt a convolution-style layout that preserves streaming flow while ensuring alignment for block-wise matching. At the semantic-level, which corresponds to the **token level**, *Focus* integrates into the attention layer to identify and retain the most relevant tokens based on cross-modal attention scores. Through dedicated scheduling, it performs token selection in a streaming fashion, enabling compression prior to memory write-back without stalling GEMM execution.

Focus operates as a standalone module, similar to pooling or activation, without interfering with the core computation pipeline. Its modularity enables broad applicability and scalability while maintaining high compression efficiency. By co-optimizing the algorithm and architecture, *Focus* achieves up to **5.0×** reduction in computation and **4.9×** reduction in memory footprint for VLM inference. Occupying only **2.7%** of the systolic array area, it is lightweight and well-suited for edge deployment. Our contributions are as follows:

- We propose multilevel concentration, a hardware-oriented redundancy removal paradigm that eliminates semantic-, block-, and vector-level redundancy in VLMs.
- We develop a co-designed streaming concentration architecture that aligns with tiling-based execution and memory access patterns with minimal hardware overhead.
- To the best of our knowledge, *Focus* is the first architecture tailored for VLMs, delivering $2.60\times/2.35\times$ performance and $2.98\times/3.29\times$ energy efficiency gains over AdapTiV and CMC, respectively.

II. BACKGROUND

A. Vision-Language Models

The success of Large Language Models (LLMs), such as GPT [47] and LLaMA [58], has driven remarkable progress across a broad range of applications. Building upon this foundation, Vision-Language Models (VLMs) extend the capabilities of LLMs to multimodal inputs, enabling joint reasoning over visual and textual information. This multimodal capability significantly broadens their utility in tasks such as video captioning [45], visual question answering (VQA) [13], [55], and interactive multimodal assistants [5], [41]. With superior adaptability and generalization in open-world visual scenarios, VLMs are emerging as a transformative technology with far-reaching impact in both academic [1], [21], [40] and industrial [2], [47], [57] domains.

Modern VLMs consist of a vision encoder and a Large Language Model (LLM) that jointly process visual and textual inputs. In video-based VLMs, videos are sampled into frames, divided into patches, and tokenized by the vision encoder into embeddings, which are projected into the LLM's word embedding space for multimodal fusion. These visual tokens are concatenated with text prompts and processed by the LLM to generate text outputs.

The LLM with Transformer [6], [59] model architecture dominates both model size and computation. For example, in LLaVA-OneVision-72B [35], it accounts for **99.35%** of

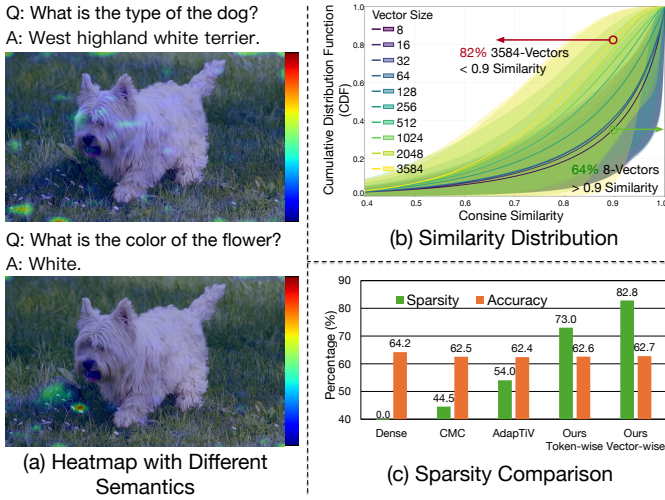


Fig. 2. Motivation for multilevel concentration. (a) Prompt-aware attention heatmaps. (b) Cosine similarity CDFs. (c) Sparsity Comparison.

parameters and **98.98%** of operations. Moreover, visual tokens typically make up 98%–99% of total inputs; in LLaVA-OneVision on the VideoMME dataset [19], each sample averages 6,272 visual tokens versus only 109 text tokens. Therefore, optimizing LLM efficiency is crucial for accelerating VLMs.

B. Efficiency Optimizations for VLM

Efficient Algorithms. Video-based VLMs generate a large number of tokens, placing heavy demands on compute and memory. To mitigate this, various token pruning techniques have been proposed [7], [28], [43], [62], [69]. For instance, Prumerge [52] uses sparse attention scores between the class token and visual tokens to discard less important ones, while FrameFusion [20] merges temporally redundant tokens across frames.

These methods show that only a small subset of tokens is needed to preserve performance. However, they often incur runtime overhead for importance estimation and produce irregular sparsity patterns that limit GPU utilization.

Hardware Accelerators. Dedicated VLM accelerators are still rare, though Vision Transformer (ViT) accelerators [16], [56], [70] offer transferable insights. AdapTiV [70] merges nearby tokens using lightweight similarity checks based on sign bits, while CMC [56] leverages video codec hardware to detect inter-frame redundancy. These designs offload token selection to specialized logic, reducing overhead and enabling efficient sparsity utilization, but are limited to coarse-grained, token-level pruning.

In contrast, *Focus* captures both coarse- and fine-grained redundancy through a multi-level concentration strategy. This broadens the scope of efficiency gains and enables hardware-friendly sparsity, as detailed in the following sections.

III. MOTIVATION

This section presents our motivation from both algorithmic and architectural perspectives across three levels:

- (1) *Token (semantic) level* prunes irrelevant tokens based on language context through semantic concentration;
- (2) *Block level (similarity scope)* detects local spatiotemporal redundancy within adjacent regions using block-wise comparison;
- (3) *Vector level (similarity granularity)* captures fine-grained sub-token redundancy via vector-wise similarity.

A. Algorithm: Multilevel Concentration

Semantic Attention Shifts with the Prompt. In Vision-Language Models (VLMs), token importance is inherently tied to the input prompt. Prior pruning methods often rely on static heuristics such as saliency or token magnitude, which fail to capture prompt-specific semantic intent.

To illustrate this, we extract cross-modal attention maps averaged from all layers of the Llava-Onevision-7B [35] model under two different prompts, as shown in Fig. 2(a). When asked “What is the type of the dog?”, attention concentrates on the dog; when asked “What is the color of the flower?”, attention shifts to the lower-left corner where the flowers reside. These examples highlight that semantically relevant tokens vary greatly with the question, and static importance metrics are inadequate.

Our semantic concentration module leverages cross-modal attention to prune uninformative tokens early, improving efficiency without degrading accuracy.

Fine-grained Granularity Enhances Redundancy Detection. Global token-level matching is often too coarse to capture redundancy arising from motion, deformation, or soft spatial shifts. As illustrated in Fig. 1(c), a token in one frame may partially overlap with several neighboring tokens in the next frame, rendering single-token matching ineffective.

To better capture such partial alignments, we divide token embeddings into vectors and perform similarity comparisons at the vector level. We extract all layers’ input from Llava-OneVision [35] model with the MLVU [76] dataset. Fig. 2(b) shows the average cosine similarity distribution across all layers, along with the variation range among layers. On average, over 64% of 8-dimensional vectors exceed a cosine similarity threshold of 0.9, compared to only 18% for 3584-dimensional vectors, indicating that finer granularity reveals substantially more redundancy. This enables higher sparsity without degrading accuracy. As shown in Fig. 2(c), our vector-level method achieves 82.8% sparsity on Llava-Video [74] with the VideoMME [19] dataset, outperforming both CMC and AdapTiV, and exceeding our token-wise variant by 9.8%. This translates to a $1.6\times$ reduction in computation, while slightly improving accuracy.

Block- and vector-level strategies are complementary: block granularity defines *where* comparisons are applied (e.g., within spatiotemporal windows), while vector granularity determines *how fine* those comparisons are conducted. Together, they yield

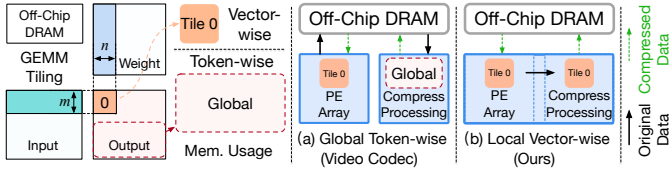


Fig. 3. (a) Global token-wise methods (e.g., CMC) perform compression off-chip after writing all token outputs to DRAM. (b) *Focus* compresses locally and on-chip at the vector level, immediately after each tile is produced.

structured sparsity that aligns naturally with GEMM tiling, enabling efficient and accurate compression in hardware.

B. Architecture: Hardware-Oriented Design

While many efforts aim to improve VLM efficiency [4], [20], [44], most focus on algorithmic techniques while overlooking hardware constraints. In high-throughput systems, algorithm and architecture must be co-designed, otherwise, even efficient algorithms can suffer from memory bottlenecks or poor data locality. As shown in Fig. 3, our design bridges this gap through a **vector-wise compression strategy** that improves both accuracy and system efficiency.

Limitations of Global Token-Wise Methods. Prior designs like CMC [56] adopt global, token-wise compression by offloading redundancy removal to a codec unit after writing full token outputs to DRAM (Fig. 3a). This incurs high bandwidth usage and sacrifices data locality. AdapTiV [70] integrates token merging into hardware, but still relies on coarse token-pair operations and must transfer uncompressed tokens before processing. If prior designs were required to perform compression before writing back to DRAM, they would need an additional large buffer; for example, CMC uses up to 1.4MB. Token-wise methods also require full-token readiness before redundancy detection, limiting streaming.

Moreover, these approaches overlook sub-token redundancy and operate at the full GEMM level, which misaligns with the execution model of systolic arrays that process small, regular GEMM tiles. Their global execution prevents fine-grained scheduling and increases memory pressure. As shown in Sec. VII-F, CMC achieves 46% sparsity but still incurs 79% of dense DRAM traffic, whereas *Focus* reaches 81% sparsity with only 21% of the bandwidth, highlighting the advantage of hardware-aligned, vector-level concentration.

GEMM-Tile Friendly Compression. *Focus* performs compression entirely within each GEMM tile, aligning with the compute flow of systolic arrays. As shown in Fig. 3(b), vector-level similarity is computed immediately after generating each $m \times n$ tile, using on-chip logic with no off-chip access. This tile-local design preserves output regularity, introduces structured sparsity, and minimizes control and data movement overhead, making it naturally hardware-efficient.

We further adopt a block-wise scheduling strategy using sliding windows. Each block is processed in-stream, enabling local reuse and eliminating the need for global buffering. Our conflict-free memory layout (Sec. VI-B) supports parallel

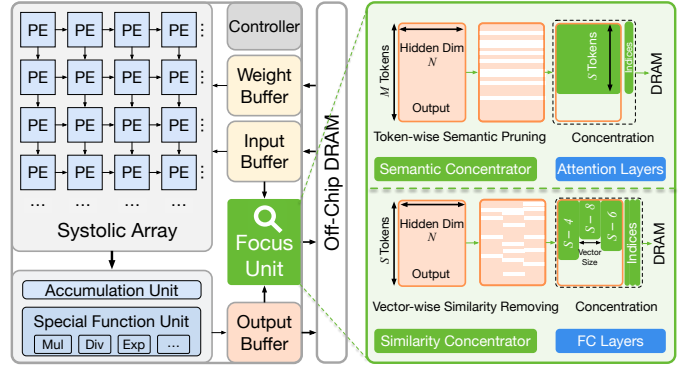


Fig. 4. Overview of the *Focus* architecture. The *Focus* Unit integrates a Semantic Concentrator (SEC) and a Similarity Concentrator (SIC), positioned between compute stages to eliminate redundancy before memory write-back. Both modules operate in a streaming manner and run entirely on-chip.

compression units without access contention, allowing *Focus* to scale with tile throughput at negligible latency.

In summary, *Focus* demonstrates effective hardware-algorithm co-optimization. Our vector-wise design improves redundancy detection and model fidelity, while streaming and tile-local execution ensure high hardware efficiency. This tightly integrated architecture makes *Focus* scalable, practical, and deployable for real-world VLM applications.

IV. Focus ARCHITECTURE OVERVIEW

Focus introduces a modular **Focus Unit** to improve compute and memory efficiency in VLMs. As shown in Fig. 4, the *Focus* Unit is integrated near the memory interface of a standard systolic-array accelerator, intercepting data between compute stages without altering the core compute pipeline. The *Focus* Unit consists of two streaming submodules:

- **Semantic Concentrator (SEC):** Performs token-level pruning in attention layers based on cross-modal Attention scores.
- **Similarity Concentrator (SIC):** Performs vector-level redundancy elimination in fully connected (FC) layers, aligned with GEMM tiling.

SEC reduces the image token sequence length from M to S . It evaluates token importance using existing attention maps and prunes low-relevance tokens early in the pipeline. Pruned tokens remain excluded in downstream layers, yielding cumulative savings in computation and memory access.

SIC further eliminates fine-grained redundancy among vectors within each GEMM tile. It compares incoming vectors in a convolution-style window and replaces similar ones with index references to shared representatives. This reduces the number of vectors processed per tile while preserving correctness via index-based reconstruction.

Both SEC and SIC operate entirely on-chip, support streaming dataflow, and dynamically adapt to data sparsity. By targeting complementary forms of redundancy from semantic and structural, *Focus* delivers efficient and scalable acceleration for Vision-Language Models. We detail the hardware implementation of SEC and SIC in Sections V and VI, respectively.

V. SEMANTIC CONCENTRATOR

The **Semantic Concentrator (SEC)** enhances inference efficiency by selectively retaining semantically important visual tokens based on language context. It operates in the attention layers and consists of three tightly coordinated yet modular components, as shown in Fig. 5: The **importance analyzer** that estimates the importance of visual tokens based on cross-modal attention. A lightweight **top- k sorter** that identifies the most important image tokens on the fly. An **offset encoder** that enables lossless index tracking for streaming token recovery.

A. Streaming Importance Analyzer

The SEC integrates directly into the attention $\text{Softmax}(QK^T)$ computation pipeline. As shown in Fig. 5(1), for each attention head, it compute a $\text{Softmax}(QK^T)$ matrix containing four blocks: image-to-image ($M \times M$), image-to-text, text-to-image ($T \times M$), and text-to-text. We extract the *Text-to-Image* block ($T \times M$) as the cross-modal importance matrix I , where M and T represent the number of image and text tokens, respectively. To estimate the importance of each image token j over n heads, we compute the maximum attention score it receives from any text token and all heads: $s_j = \max_{1 \leq k \leq n} \max_{1 \leq i \leq T} I_{i,j}^{(k)}$. This results in an importance vector of shape $1 \times M$ across all heads. An on-chip buffer of 25 KB is used to store the importance vector.

As depicted in Fig. 5(2), the importance analyzer uses a parallel max units to process the output of the attention SoftMax (provided by the special function unit). To match throughput, a max units processes a attention scores concurrently. This streaming design supports two dataflows: **Parallel (spatial) stream**: Attention columns are streamed directly into max units. **Orthogonal (temporal) stream**: Attention rows are buffered locally, enabling column-wise reduction.

This fully streaming design ensures minimal area and latency overhead. Since no global operations are needed, the analyzer is decoupled from the main compute path and incurs negligible runtime cost.

B. Top- k Bubble Sorter

Once the $1 \times M$ importance vector is computed, the system must identify the top- k most relevant tokens. To avoid sorting all M tokens globally, SEC adopts a pipelined bubble sorter as shown in Fig. 5(4). By chaining the a max units used earlier, we construct an a -way streaming bubble sorter. This structure incrementally refines the top- a tokens, allowing us to compute top- k selection over the M candidates in $\frac{M \cdot k}{a}$ cycles, substantially more efficient than full sorting.

Crucially, this process is fully overlapped with the computation of image attention ($S^{(image)} = Q^{(image)} K^T$), which dominates the overall runtime. Let the image attention (within QK^T GEMM) require $\frac{M \cdot (M+T) \cdot h \cdot n}{a \cdot b}$ cycles, where h is the head dimension, n is the number of heads, and $a \times b$ is the PE array size. The ratio of attention to the sorting operation is: $\frac{M \cdot (M+T) \cdot h \cdot n}{a \cdot b} \cdot \frac{a}{M \cdot k} = \frac{(M+T) \cdot h \cdot n}{k \cdot b}$. In typical configurations,

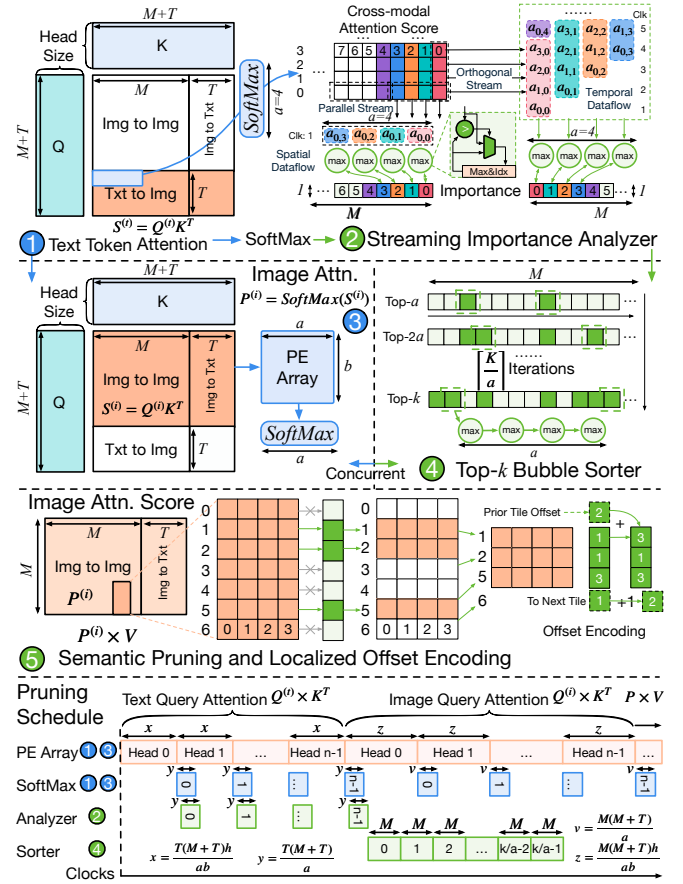


Fig. 5. Overview of the Semantic Concentrator (SEC), including the streaming importance analyzer, top- k sorter, and offset encoder.

$h \cdot n$ reaches into the thousands (e.g., 3584), while b is much smaller (e.g., 32) and $k < (M + T)$. Therefore, the sorting operation completes well before the $Q^{(i)} K^T$ finishes, ensuring that the SEC remains off the critical path and introduces no runtime bottleneck. A scheduling diagram is shown in Fig. 5 bottom to better understand the overlapping.

C. Semantic Pruning and Offset Encoding

After identifying the top- k most important tokens, we apply **semantic pruning** for the input of the subsequent attention operation, $P^{(i)} \times V$. As shown in Fig. 5(5), only the retained tokens are loaded and processed in $P^{(i)} \times V$, eliminating the need for any memory access or computation on pruned tokens.

In pure pruning mode, no additional metadata is needed. However, to support later stages (e.g., similarity concentration), we must record the position of retained tokens for spatial-temporal information. For this, the SEC generates **localized offset encodings**. The offset encoder, shown in Fig. 5(5), operates in a sliding window over the retained tokens. For each token, it records a small integer representing its offset to the previous token. This compact encoding is sufficient to restore positional alignment for future operations (e.g., similarity matching). The encoder's computation is local

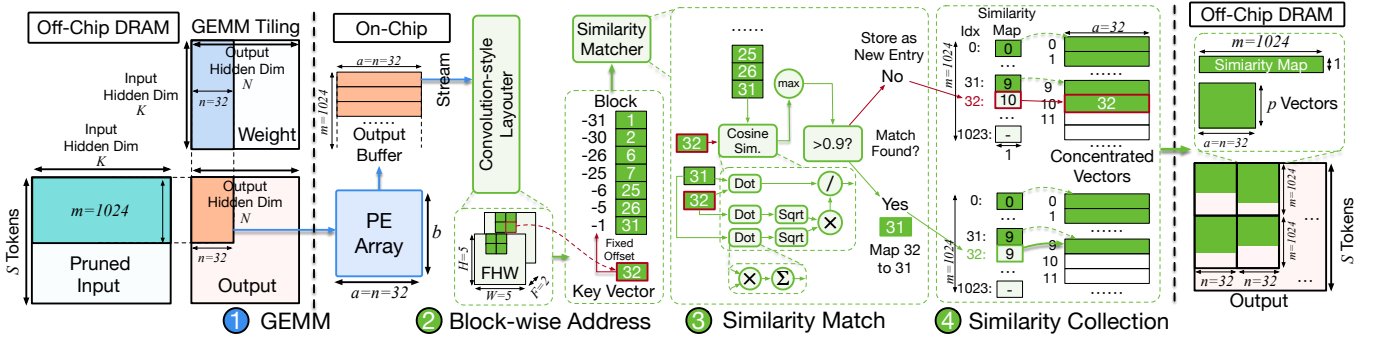


Fig. 6. Overview of the Similarity Gather module. (1) GEMM tiling. (2) Convolution-style layout reorders outputs into a block-wise structure. (3) Cosine similarity is computed within blocks to detect and eliminate duplicates. (4) Only unique vectors are stored, while a similarity map enables reconstruction.

and streaming, requiring only lightweight registers and no global memory access.

Overall, the entire Semantic Concentrator, including the analyzer, sorter, and encoder, is fully modular and incurs minimal overhead. SEC selectively retains the most informative visual tokens using cross-modal attention, top- k selection, and compact position encoding. It operates transparently within the attention, requires no additional global memory access, and introduces negligible runtime or area overhead.

VI. SIMILARITY CONCENTRATOR

After semantic-level token pruning, subsequent FC layers operate seamlessly on the reduced token set, as the pruning is token-aligned and preserves structural layout. However, semantic pruning alone does not address fine-grained redundancy at the vector level. In contrast, **Similarity Concentration** targets vector-wise redundancy by matching and merging similar output vectors within local regions across frames.

Different from semantic pruning, where unimportant tokens are discarded, similar vectors often carry essential information and thus require accurate removal and later reconstruction. To support this, the similarity process includes two core components: **Similarity Gather**: removes similar vectors and constructs a compact output. **Similarity Scatter**: restores the original full layout using a similarity map.

A. Similarity Gather

In this section, we first detail the design of **Similarity Gather**, which efficiently operates in a fully streaming fashion.

GEMM Tiling. As shown in Fig. 6(1), Similarity Gather operates on the output of GEMM¹. Assume the input has dimension $M \times K$, the weight matrix is $K \times N$, and the output is $M \times N$. Due to limited on-chip resources, we adopt a tiling strategy widely used in modern accelerators [8], [32]. Specifically, the input and weight tiles are of size $m \times K$ and $K \times n$, and the output tile is $m \times n$, where we typically set $m = 1024$ and $n = 32$. The PE array performs one output tile at a time, producing a -length output vectors, where $a = n = 32$ in our implementation. These vectors are then

streamed to the Similarity Gather logic for processing when an output tile gets ready.

Block-wise Addressing. To exploit spatiotemporal redundancy, we adopt a **convolution-style layout** over two adjacent frames, as illustrated in Fig. 6(2). A $2 \times 2 \times 2$ sliding window spans both spatial and temporal dimensions with a stride of 1, forming a block that contains 8 vectors, 4 from each frame. Each element in the block is an a -dimensional output vector produced by the GEMM operation, and the block serves as a localized comparison group for redundancy detection.

To efficiently support this structure, GEMM outputs are dynamically reorganized into the convolution-style layout using a dedicated reordering module, as detailed in Sec. VI-B. Within each block, the vector with the highest index (e.g., token ID 32) is selected as the key vector and compared against the other 7 vectors in the same block (e.g., token IDs 1 through 31) to identify potential redundancy.

Vector-wise Similarity Matching. As shown in Fig. 6(3), each key vector is streamed into the **Similarity Matcher**, which performs localized comparisons to determine whether the vector is redundant. We adopt cosine similarity to compare two vectors $\mathbf{p}$ and $\mathbf{q}$ of length 32:

$$\frac{\mathbf{p} \cdot \mathbf{q}}{\|\mathbf{p}\| \cdot \|\mathbf{q}\|} = \frac{\sum_{i=1}^{32} p_i q_i}{\sqrt{\sum_{i=1}^{32} p_i^2} \cdot \sqrt{\sum_{i=1}^{32} q_i^2}}.$$

Thanks to the regularity of the convolution-style layout, each token can precompute its L2-norm ($\|\mathbf{p}\|$) and store it in a buffer. This allows the matcher to perform similarity comparisons using only a single dot-product unit and a small number of low-overhead element-wise operations. Furthermore, the vector-level granularity reduces the normalization length to 32, greatly simplifying the hardware design compared to token-wise similarity matching. In contrast, prior accelerators such as AdapTiV [70] and CMC [56] compute similarity at the token level, requiring expensive global memory access and full-sequence comparisons. By operating at the vector level within localized blocks, *Focus* achieves significantly lower matching overhead while maintaining semantic fidelity.

In practice, most of these operations are already supported by the Special Function Unit (SFU), which is commonly used for *RMSNorm* [72] and *SoftMax* computations. Com-

¹Similarity Gather on output of FFN, O projection, and PV GEMM

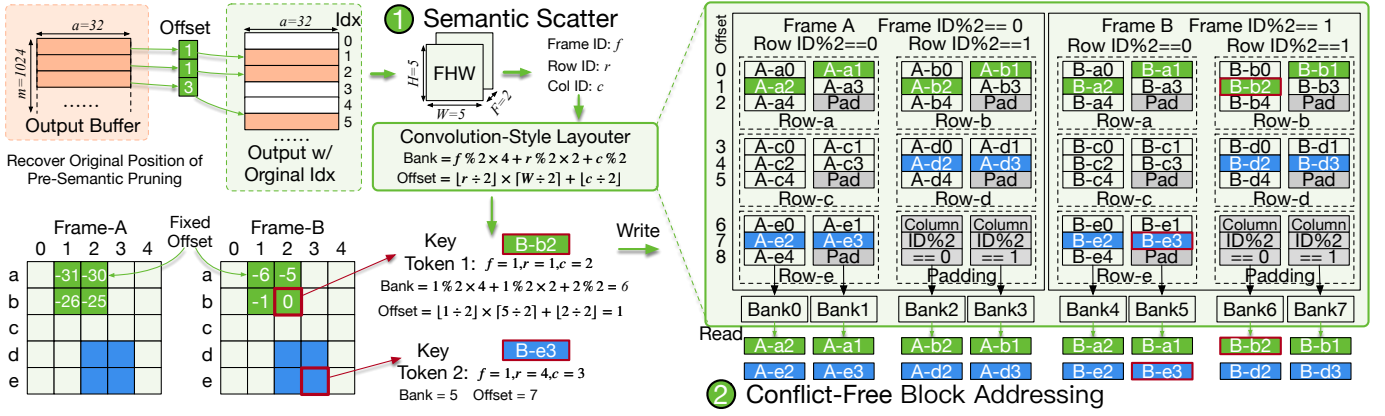


Fig. 7. The convolution-style layouter enables accurate token positioning and conflict-free memory access for block-level similarity. (1) Reconstructs token positions via semantic offsets and maps outputs to a FHW-layout. (2) Enables conflict-free addressing across memory banks without data duplication.

pared to these more complex operations, cosine similarity is lightweight and well-suited for hardware acceleration. Although the matcher could reuse existing SFU logic, for fairness, we implement it as a separate module and include its area and energy in our evaluation. The total overhead remains minimal, accounting for $<1\%$ of the systolic array design.

It is worth noting that similarity matching is *not* on the critical path of GEMM, as comparisons are performed only once per output tile. For a tile with $m = 1024$ vectors, each requiring 7 pairwise comparisons and 1 L2-norm computation (based on the $2 \times 2 \times 2$ block structure), the matcher needs at most $8 \times m$ cycles to process the tile. In contrast, GEMM requires $\frac{K}{b} \times m$ cycles, where K is the hidden dimension and b is the number of PE rows. In our setup, with $K = 3584$ and $b = 32$, GEMM takes $112 \times m$ cycles per tile, far exceeding the cost of similarity matching. Only when $K < 256$ does the matcher approach the critical path.

To address this corner case, we can scale the design by deploying multiple matcher units in parallel. Our convolution-style layout inherently supports conflict-free parallel access, allowing similarity matching to be fully overlapped with GEMM computation without introducing additional latency.

Similarity Collection. Once similarity matching completes, each vector has two outcomes: No match: The vector is unique and added to the concentrated output buffer. Match found: The vector matches a previously stored one (e.g., token 32 matches token 31), and we reuse the index of the matched token.

To support lossless reconstruction, we maintain a **Similarity Map** of size $1 \times m$ per tile. This map records, for each of the original m output vectors, the index of its representative in the compact buffer. For instance, if token 32 matches token 31, we assign token 32 the index “9” from token 31. After processing all $m = 1024$ vectors in a tile, only the deduplicated vectors and the similarity map are written back to DRAM. This significantly reduces memory bandwidth and storage.

All stages of this pipeline, including reordering, matching, and mapping, are performed on-chip, in a streaming fashion, without global synchronization or off-chip overhead. This

localized similarity removal aligns naturally with GEMM tiling and preserves high data locality throughout execution.

B. Convolution-style Layouter

We now describe the design of the *convolution-style layouter*, which addresses two key challenges in enabling efficient block-level similarity matching after semantic pruning: (1) recovering token positions and (2) avoiding memory access conflicts during parallel execution.

Challenge 1: Recovering Token Positions after Pruning. Semantic pruning disrupts the spatial structure of tokens by removing unimportant entries, making it nontrivial to identify the 2D position of retained tokens in the original frame. To enable meaningful $2 \times 2 \times 2$ comparisons across adjacent frames, we must reconstruct each token’s (Frame, Height, Width) coordinate after pruning.

As shown in Fig. 7(1), we achieve this using the *offset encoding* generated during the semantic pruning stage (see Sec. V-C). This offset, streamed alongside the GEMM output, allows us to recover the original spatial location of each token. Tokens are then reorganized into a structured 3D tensor layout following the FHW (Frame–Height–Width) order to support localized block grouping.

Challenge 2: Avoiding Memory Conflicts in Parallel Matching. To form a $2 \times 2 \times 2$ spatiotemporal block, vectors are drawn from multiple rows, columns, and frames. A naive layout may introduce bank conflicts or require data duplication across SRAM banks, an approach used by traditional CNN accelerators [8] but with significant memory overhead (up to $8 \times$ replication).

To eliminate these conflicts, we propose a **conflict-free convolution-style layout**, shown in Fig. 7(2), which deterministically maps each token to a unique bank and offset based on its FHW position. Given frame index f , row r , and column c , the memory bank and address are computed as:

$$\text{Bank} = f \bmod 2 \times 4 + r \bmod 2 \times 2 + c \bmod 2,$$

$$\text{Offset} = \left\lfloor \frac{r}{2} \right\rfloor \times \left\lfloor \frac{W}{2} \right\rfloor + \left\lfloor \frac{c}{2} \right\rfloor,$$

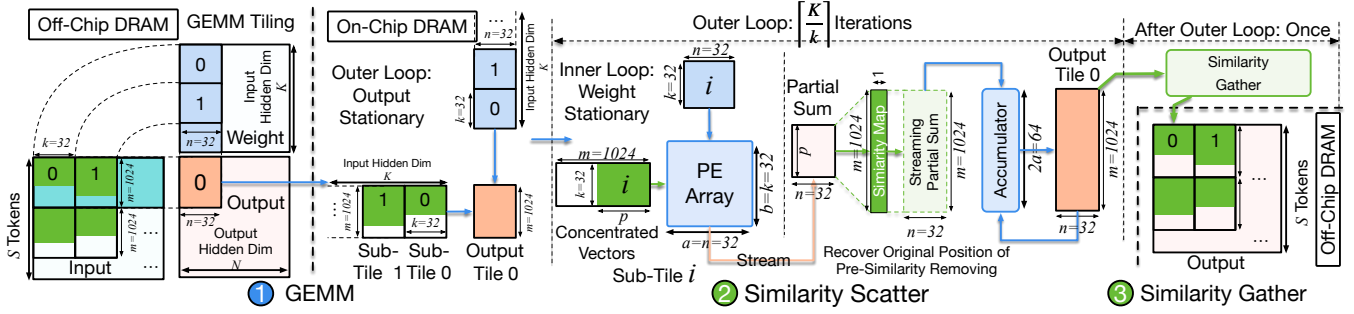


Fig. 8. GEMM tiling and Similarity Scatter design. (1) GEMM computes over concentrated vectors. (2) Similarity Scatter reconstructs and accumulates vector results using similarity maps. (3) Final output is passed to Similarity Gather once after all iterations in a tile.

where W is the width of the frame. This mapping guarantees that all 8 vectors in any $2 \times 2 \times 2$ block reside in distinct memory banks and can be read simultaneously without contention.

Key Insight: Unlike traditional approaches that duplicate inputs to avoid access conflicts, our layout achieves fully parallel, conflict-free access *without any data replication*. This enables streaming similarity matchers to operate in parallel across tiles and spatial regions, scaling throughput without modifying the GEMM pipeline. The layouter thus plays a critical role in supporting parallel similarity execution and maintaining high utilization.

C. Similarity Scatter

GEMM Tiling for Concentrated Vectors. As shown in Fig. 8(1), Similarity Scatter operates on the concentrated vectors generated from earlier stages. Since only a subset of the original $m = 1024$ tokens is retained in each tile ($p < 1024$), the input to this GEMM stage is **logically sparse** but **structurally dense**. To maintain compatibility with standard systolic-array architectures, GEMM is performed using a conventional tiling scheme with dimensions $m = 1024$ and $n = 32$. The GEMM execution follows a two-level nested loop structure: The **outer loop** adopts an *output-stationary* dataflow, keeping the $m \times n$ output tile resident on-chip to accumulate results across the K dimension. The **inner loop** follows a *weight-stationary* strategy, loading one $k \times n$ weight sub-tile into the PE array while streaming in a $p \times k$ sub-tile of concentrated input vectors.

Each inner loop iteration computes partial products and generates one a -dimensional partial sum vector per cycle. Our vector size 32 matches with the k tile size and array height b , ensuring full utilization of the PE array. These vectors are streamed out and accumulated over successive iterations to form the final tile result. The key advantage arises from the reduced number of active input vectors ($p < 1024$), which significantly lowers the computational workload. However, since different sub-tiles may have different subsets of concentrated vectors, each possibly representing multiple original tokens, direct accumulation would produce incorrect outputs due to semantic aliasing.

Similarity Scatter and Gather. To resolve this, we introduce the **Similarity Scatter** module, illustrated in Fig. 8(2).

After each GEMM step, the generated partial sums are streamed into a temporary buffer. Using the similarity map from previous layer's the gather phase (see Sec. VI-A), each partial sum is replicated and redistributed to its associated original token indices, reconstructing the full $m = 1024$ output. This scattered output is then accumulated into an output-stationary buffer spanning all outer loop iterations. To maintain throughput parity, we employ a $2a$ -wide accumulator (e.g., 64 when $a = 32$), enabling concurrent accumulation of reconstructed vectors and streaming outputs. The reconstruction process is performed in-place, incurs negligible overhead, and does not require additional memory allocation.

Upon completing all $\lceil \frac{K}{k} \rceil$ outer loop iterations, the fully accumulated output tile is passed to the **Similarity Gather** unit (see Sec. VI-A), shown in Fig. 8(3). This final stage is invoked only once per tile after GEMM concludes and lies entirely off the critical compute path.

In summary, by executing GEMM on a compact set of concentrated vectors, *Focus* achieves substantial compute savings. Through the Similarity Scatter module, it efficiently reconstructs full output tiles with minimal accuracy loss, and the final gather stage removes vector-level redundancy. This hardware-oriented, vector-granular compression strategy ensures high compute efficiency while preserving model fidelity. Our evaluation shows that the additional logic is lightweight and does not impact GEMM throughput, making it a key enabler of *Focus*'s performance advantage.

VII. EVALUATION

A. Methodology

Evaluation Models and Datasets. We evaluate *Focus* using three representative VLMs with video understanding and reasoning capabilities: Llava-OneVision-7B (Llava-OV) [35], Llava-Video-7B (Llava-Vid) [74], and MiniCPMV-2.6 (MiniCPM) [68]. These models are tested on three widely adopted video understanding benchmarks: VideoMME (VMME) [19], MVBench (MVB) [37], and MLVU [76]. These datasets include diverse video types and durations, enabling a holistic evaluation of model capabilities across comprehension, temporal reasoning, and multimodal alignment. We use open-source models obtained from HuggingFace Transformers [66]

and perform evaluation via the `lmms-eval` [73] multimodal benchmarking framework to ensure consistency and fairness.

Baselines. We compare *Focus* against two state-of-the-art architectures: AdapTiV [70], a vision transformer accelerator, and CMC [56], an accelerator optimized for video transformers. We extend their designs to make them compatible with VLMs. CMC performs inter-frame similarity checks, whereas AdapTiV focuses on intra-frame similarity detection; both exclude text tokens. We also compare with the vanilla systolic array [34] architecture for a base reference. In addition to hardware baselines, we also compare with FrameFusion [20], a state-of-the-art token pruning algorithm tailored for efficient VLMs with video inputs.

Algorithm Implementation of *Focus* and Baselines. We implement the algorithm of our proposed *Focus* method in PyTorch [48]. For the baselines, we faithfully reproduce the token pruning algorithm from AdapTiV and CMC, carefully tuning their hyperparameters for application to VLMs. For FrameFusion, we adopt the official open-source implementation without modification. All algorithms are executed on an NVIDIA A100 GPU [46] using FP16 precision for fair and consistent comparison.

Architecture Implementation of *Focus* and Baselines. Our *Focus* architecture setup is shown in Tbl. I. To evaluate architectural performance, we develop a cycle-accurate simulation framework based on SCALEsim-v2 [51]. The simulator accepts layer-wise sparse traces generated from specific models and datasets in our PyTorch implementation, enabling precise modeling of cycles and memory access. We implement the *Focus* architecture in SystemVerilog and generate the on-chip SRAMs using the TSMC N28HPC+ Memory Compiler. The RTL is synthesized with a target clock period of 1.32 ns (≈ 757 MHz) under the worst-case slow-slow (SS) corner (0.81V, 125°C), achieving 0 ns worst negative slack (WNS) and providing a 34% timing margin for place-and-route at 500 MHz. The resulting area is reported from post-synthesis analysis, and the on-chip power is obtained from post-synthesis simulation using Synopsys Design Compiler. Off-chip DRAM energy is modeled with DRAMsim3 [38] for device-level power. For a fair comparison, we also implement the core logic of all baseline accelerators in SystemVerilog and evaluate their area and energy using the same toolchain as *Focus*.

TABLE I
Focus ARCHITECTURE SETUP

PE Array	32×32 ; FP16 Mul FP32 Acc; Weight Stationary
<i>Focus</i> Hyper-params	Block Size: $2 \times 2 \times 2$; Vector Length: 32; Similarity Threshold: 0.9; M Tile Size: 1024 Semantic: Retain 40%/30%/20%/15%/10% of total image tokens at layer 3/6/9/18/26
On-Chip Buffer	Input: 128KB; Weight: 78KB; Output: 512KB; Layouter Buffer: 16KB for 256-vector window; 734KB in total.
Off-Chip Memory	DDR4 4Gb $\times$ 16, 2133R, 4 Channels, 64GB/s

TABLE II
ACCURACY AND COMPUTATION SPARSITY OF *Focus* AND BASELINES

Models	Dataset	Metric	Ori.	FF	Ada.	CMC	Ours
Llava-Vid	VMME	Acc. Sparsity	64.15 00.00	62.00 70.00	62.44 52.15	62.52 58.62	62.74 82.82
	MLVU	Acc. Sparsity	67.74 00.00	65.38 70.00	65.94 32.52	65.17 42.46	65.99 78.26
	MVB	Acc. Sparsity	60.33 00.00	57.20 70.00	57.73 41.07	58.18 53.00	58.20 78.44
Llava-OV	VMME	Acc. Sparsity	58.41 00.00	57.70 70.00	58.33 36.80	58.11 47.95	58.70 81.49
	MLVU	Acc. Sparsity	63.32 00.00	62.54 70.00	62.22 39.55	62.50 35.48	62.52 78.34
	MVB	Acc. Sparsity	58.38 00.00	56.93 70.00	56.83 42.03	56.75 63.69	56.78 85.49
MiniCPM	VMME	Acc. Sparsity	58.81 00.00	58.81 70.00	58.07 49.27	55.89 57.20	58.30 82.87
	MLVU	Acc. Sparsity	55.89 00.00	54.80 70.00	54.84 41.88	43.80 35.23	53.59 78.01
	MVB	Acc. Sparsity	55.63 00.00	52.43 70.00	53.70 50.09	48.78 40.27	54.30 75.99

B. Algorithmic Accuracy and Theoretical Sparsity

To evaluate the effectiveness of the multilevel concentration technique in *Focus*, we compare both model accuracy and the achieved computational sparsity against baseline methods. The computation sparsity is calculated through the ratio of the number of operations using the method to the number of operations required by the systolic array with original input. The results are presented in Tbl. II.

Focus consistently achieves the highest accuracy across most evaluated scenarios, outperforming both software-only methods and hardware-based approaches. Compared to the original, uncompressed models, the average accuracy degradation with *Focus* is only 1.20%, demonstrating its ability to preserve semantic fidelity.

In addition to maintaining high accuracy, *Focus* also achieves the highest computational sparsity across all models and datasets. Specifically, *Focus* achieve sparsity of **80.19%** on average, delivering 37.37% and 31.98% higher sparsity than AdapTiV and CMC, respectively, and outperforms FrameFusion by over 10.19% in sparsity.

C. Performance and Energy Evaluation

We compare *Focus* against baseline methods, including the vanilla systolic array (SA), Adaptiv, and CMC, across multiple VLM models and datasets. The architectural setup for all baselines and *Focus* is detailed in Tbl. III. We maintain the same frequency, technology node, number of processing elements, operand bit width, and DRAM bandwidth across all designs. We further compare against the performance on an NVIDIA Jetson Orin Nano GPU [12], evaluated with and without the FrameFusion algorithm. The performance and energy results are presented in Fig. 9 (a) and (b).

Performance. *Focus* achieves significant performance improvements across all benchmarks. On average, it delivers a $4.47\times$ speedup over the vanilla systolic array, which process

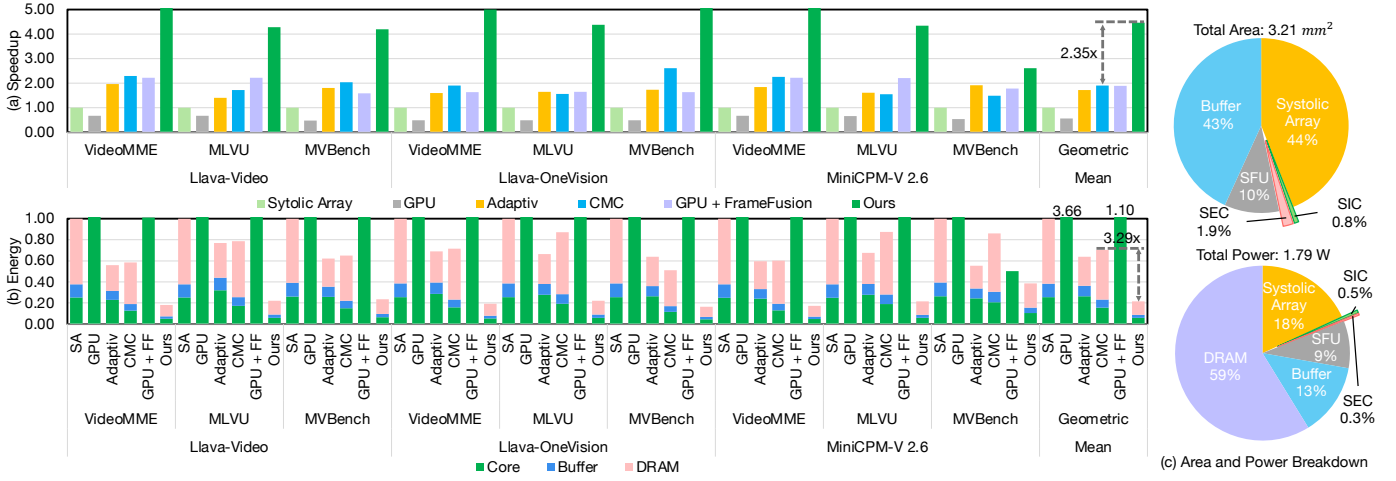


Fig. 9. Left: Speedup and Energy efficiency. Right: Area and power breakdown

dense input. This improvement stems from the ability of multi-level concentration to aggressively compress input tokens, getting 80.2% sparsity.

Compared to AdapTiV, *Focus* achieves a **2.60×** average speedup. While AdapTiV effectively detects and prunes nearby redundant visual tokens, it operates at a coarser granularity. In contrast, *Focus* performs vector-wise similarity removal, enabling finer-grained redundancy elimination.

Against CMC, *Focus* achieves a **2.35×** speedup. While CMC leverages external video codecs to perform wide-range redundancy search, this approach is often inefficient due to a high rate of mismatches. In contrast, *Focus* efficiently identifies sufficient redundancy within localized blocks using its on-chip similarity matcher.

Compared with the GPU, our design achieves a **7.90×** speedup over the GPU and a **2.37×** speedup over the GPU running with FrameFusion. This improvement stems from our architecture’s ability to achieve higher computational utilization than the GPU. Moreover, *Focus* attains higher sparsity than FrameFusion due to its finer-grained redundancy removal, which is difficult to exploit on GPU Tensor Cores.

Energy Efficiency. As shown in Fig. 9(b), we report the total energy consumption of *Focus* and baseline designs, normalized to the vanilla systolic array (SA). The energy breakdown includes three components: on-chip core, on-chip buffer, and off-chip memory.

Compared to SA, GPU, AdapTiV, CMC, and GPU with FrameFusion, *Focus* achieves average energy efficiency improvements of **4.67×**, **17.09×**, **2.98×**, **3.29×**, and **5.13×**, respectively. These results highlight that *Focus* delivers significant savings across both computation and memory under constrained on-chip budget. This efficiency gain stems from our architecture’s ability to sparsify output of GEMM on-chip immediately after output tile generation, ensuring that all subsequent off-chip memory transactions operate on compressed data. A detailed analysis of memory access reduction is presented in Sec. VII-F.

TABLE III
CONFIGURATION COMPARISON OF *Focus* AND BASELINE ARCHITECTURE

Architecture	SystolicArray	Adaptiv	CMC	Ours
Technology	28nm	28nm	28nm	28nm
Frequency	500MHz	500MHz	500MHz	500MHz
PE Array	32x32 16-bit	16x64 16-bit	32x32 16-bit	32x32 16-bit
Buffer Size	734KB	768KB	907KB	734KB
DRAM Bandwidth	64GB/s	64GB/s	64GB/s	64GB/s
On-chip Area/mm ²	3.12	3.38	3.58	3.21
On-chip Power/mW	720	1176	832	736

Area and Power Analysis. The area and power consumption of *Focus* and the baselines are also summarized in Tbl. III. The power statistics is derived on Llava-Video-7B with VideoMME dataset. Our *Focus* design occupies 3.21 mm² of on-chip area and consumes 736 mW of power, both of which are lower than those of Adaptiv and CMC.

Focus is smaller than CMC, as the external video codec used by CMC incurs substantial hardware overhead. Compared to Adaptiv, which adopts a lightweight similarity detection mechanism, *Focus* remains more efficient due to its streaming SEC that operates on localized input. Despite its enhanced functionality, *Focus* introduces only a **2.7%** increase in area and a **0.9%** increase in power consumption relative to the systolic array architecture. These results highlight the efficiency and low overhead of the *Focus* unit, which delivers significant performance benefits within a modest hardware budget.

To gain a deeper understanding of the overhead introduced by *Focus*, we present a detailed breakdown in Fig. 9(c). We observe that the proposed Semantic Concentrator and Similarity Concentrator are both highly lightweight, accounting for only 1.9% and 0.8% of the overall area, respectively. These two units also contribute negligibly to the overall power consumption. This demonstrates that SEC and SIC are well-

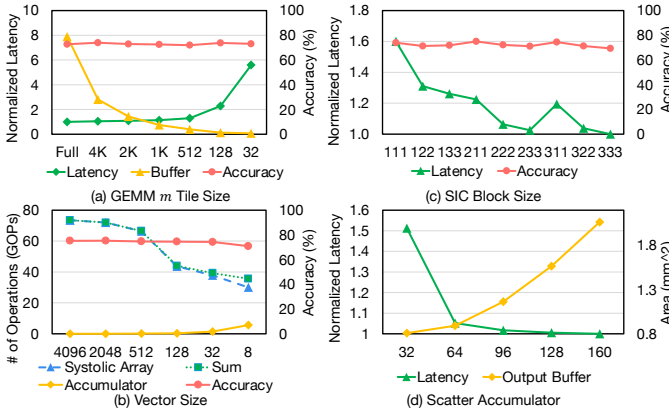


Fig. 10. Design Space Exploration

suited for resource-constrained scenarios.

Overall Insights. Beyond speedup and energy gains, *Focus* establishes a new paradigm for redundancy-aware VLM acceleration through tight algorithm–hardware co-design. At the **token level**, it performs on-the-fly *Top-k detection* via streaming processing, handling sparsity in real time with minimal cost. At the **block level**, a *block-wise sliding window* propagates local similarity using only on-chip resources, reducing memory and buffer demand. At the **vector level**, *Focus* applies *vector-wise similarity pruning* with a *gather–scatter* scheme to control fine-grained irregular access and fully exploit sparsity. Together, these techniques translate algorithmic sparsity into tangible performance gains with minor hardware complexity.

D. Design Space Exploration

To evaluate the impact of key architectural parameters in *Focus*, we conduct a comprehensive design space exploration. We focus on four primary factors, varying each individually while fixing the others to their default values to isolate their effects. Note that architectural parameters, other than the number of scatter accumulators, may also affect model accuracy. We evaluate accuracy under these variations and observe that the impact is generally negligible, allowing us to safely prioritize performance in our design exploration. All measurements are taken on the Llava-Video-7B model, using either the VideoMME or MLVU dataset.

GEMM m Tile Size. As shown in Fig. 10(a), we sweep the tile size from the full input height down to 32. As the tile size decreases, the end-to-end latency steadily increases. This trend arises because similarity gathering operates per tile. When a 2×2 block crosses tile boundaries, *Focus* only compares tokens within the same tile as the key token. For example, when the first token of a tile is the key, its neighbors outside the tile are unavailable for comparison. With smaller tile sizes (e.g., $m = 32$), such boundary-crossing cases become more frequent, causing potentially similar vectors to be treated as distinct due to the limited comparison scope.

While larger tiles offer better compression, they require more on-chip buffer to store intermediate results, increasing area and power consumption. We observe a trade-off between

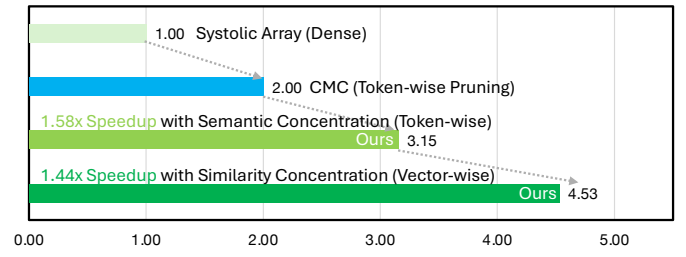


Fig. 11. Ablation Study for *Focus*

latency and buffer usage. From the latency–buffer curve in Fig. 10(a), a tile size of 1024 emerges as an optimal design point. It incurs only 19% higher latency compared to the full-height tile while substantially reducing buffer requirements to a practical level.

Vector Size. Vector size determines the granularity of similarity concentration and directly impacts the sparsity and operation counts. To assess this, we measure the number of operations of a layer in two main components of *Focus*: (1) MAC operations in the main systolic array, and (2) accumulation operations in the outer accumulator during Similarity Scatter.

As shown in Fig. 10(b), reducing the vector size leads to fewer operations in the systolic array. This is because smaller vectors enable finer-grained similarity comparisons, allowing more aggressive redundancy removal and reducing the input size to the PE array. However, smaller vector sizes also increase the number of K -dimension iterations, requiring more frequent accumulation, which in turn raises the operation count in the accumulator.

Beyond operation count, the systolic array dimension b must be equal to or less than the vector size to utilize the benefits of fine-grained input. Taking both operational efficiency and hardware compatibility into account, we identify a vector size of 32 as an optimal design point, achieving strong compression while maintaining high utilization of the systolic array.

Similarity Concentrator Block Size. The block size used in the SIC directly impacts the spatial and temporal context available for similarity detection. We vary the block size along both the temporal (frame) and spatial (height and width) dimensions to examine its impact on performance. As shown in Fig. 10(c), the three-digit labels on the x-axis denote block sizes across these dimensions (e.g., 122 indicates $f=1, h=2, w=2$). We observe that enlarging the block size in either temporal or spatial dimensions reduces latency, as larger blocks provide broader context for similarity detection. Notably, extending the block size along the temporal dimension yields a more pronounced latency reduction compared to spatial extensions, which we attribute to the strong inter-frame similarity inherent in video inputs. We find that a block size of $2 \times 2 \times 2$ is sufficient to provide strong performance.

Scatter Accumulator. The number of accumulators in similarity scatter affects throughput and pipeline efficiency. Ideally, accumulation should finish before the next output tile arrives from the systolic array. As shown in Fig. 10(d), using

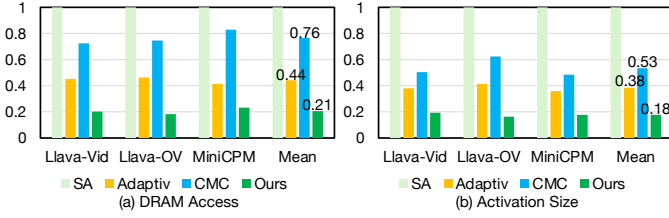


Fig. 12. Memory access analysis (a) overall DRAM access (b) activation size

64 accumulators achieves near-peak performance with only a 5% latency overhead compared to a larger 160-accumulator design, with diminishing returns beyond that point. This configuration also simplifies buffer design.

Semantic Pruning Configuration. In our Semantic Pruning scheme, the value of “k” in top-k pruning is determined by multiplying the original number of image tokens by a predefined retention ratio. We search multiple layer-wise retention configurations and select the one offering the best sparsity–accuracy trade-off, which is adopted in our design. The final setup is summarized in Tbl. I, where pruning is applied to five selected layers whose retention ratios differ from the preceding layer. Future work may further enhance this strategy by dynamically adapting to input contexts, e.g., using a post-softmax attention threshold or top-p pruning [39], though such adaptation can introduce runtime variations across inputs.

E. Ablation Study

To assess the contribution of each component in *Focus*, we perform an ablation study on Llava-Video-7B and report speedup, as shown in Fig. 11. We incrementally enable the SEC and SIC, comparing results against a dense systolic baseline and CMC [56]. When only the SEC is enabled, *Focus* achieves a $3.15\times$ speedup over the uncompressed systolic baseline and a $1.58\times$ speedup over CMC. This demonstrates that semantic-aware pruning remove a large fraction of irrelevant visual tokens based on textual guidance, outperforming prior token-pruning strategies.

Enabling the vector-wise SIC further boosts speedup by an additional $1.44\times$. This highlights the ability of SIC to exploit residual redundancy among retained tokens at a finer vector granularity, beyond what semantic pruning alone can uncover. SEC and SIC together yield a $4.53\times$ speedup over the dense baseline and $2.26\times$ over CMC, confirming the effectiveness and efficiency of the *Focus* design.

F. Memory Access

We analyze the off-chip memory traffic and average input matrix size of *Focus* compared to baseline designs. As shown in Fig. 12(a) and (b), *Focus* achieves the lowest DRAM access and input matrix size across all methods. Compared to the dense systolic array, we compress the input matrix by $5.6\times$ and reduce memory traffic by $4.9\times$.

This reduction stems from the joint effect of the SEC and SIC, which sparsifies the input at both token and vector levels.

TABLE IV
INFLUENCE OF INT8 QUANTIZATION ON ACCURACY AND SPARSITY

Models	Datasets	Dense		Ours		Ours	
		Acc.	Degrade	Acc.	Degrade	Sparsity	Degrade
Llava-Vid	VMME	64.22	-0.07	62.33	0.41	82.48	0.34
	MLVU	68.21	-0.47	64.94	1.05	78.10	0.17
	MVB	59.75	0.58	57.95	0.25	78.04	0.40
Llava-OV	VMME	58.70	-0.29	57.44	1.26	81.46	0.03
	MLVU	63.38	-0.06	62.41	0.11	78.35	-0.01
	MVB	58.55	-0.17	56.18	0.60	85.35	0.14
MiniCPM	VMME	58.63	0.18	57.96	0.34	82.84	0.03
	MLVU	55.93	-0.04	53.22	0.37	77.99	0.02
	MVB	55.13	0.50	54.03	0.27	75.97	0.02

Additionally, the output of each FC layer is immediately compressed on-chip before being written to memory, so only compressed activations are transferred to DRAM, minimizing total memory access.

Compared to both CMC and AdapTiV, *Focus* achieves significantly higher input compression and lower DRAM access: $3.0\times$ and $2.2\times$ higher compression ratios, and $3.7\times$ and $2.2\times$ DRAM traffic reduction, respectively.

CMC relies on codec-based similarity detection over wide temporal windows and full-token representations, requiring large uncompressed regions to be staged in DRAM before processing. This leads to redundant memory transfers, as data must be written and read again for similarity detection. Similarly, AdapTiV performs local token pruning but still processes on whole-token granularity. By avoiding these limitations through lightweight, streaming-compatible similarity matching, *Focus* achieves superior memory efficiency with minimal overhead.

G. Synergy with Quantization

Focus is fully compatible with standard quantization techniques. We integrate *Focus* with INT8 quantization using bitsandbytes [14], and the results in Tbl. IV show the impact on accuracy and sparsity compared to FP16. INT8 causes an average accuracy drop of 0.5% and a sparsity change of 0.13% relative to FP16. Although this loss is slightly higher than the 0.02% degradation in the dense model, it is reasonable since *Focus* and quantization jointly compress the model. Overall, the accuracy drop remains minor, and *Focus* effectively maintains its redundancy-removal capability under quantization, demonstrating strong synergy for efficient VLM inference.

VIII. DISCUSSION

A. Generalization Ability of *Focus*

We further examine the generalization ability of *Focus*. Although *Focus* is originally designed for video-based VLMs, it can be directly extended to image-based VLMs by treating a single image as a one-frame video. While temporal similarity is no longer present in this setting, substantial semantic redundancy and spatial similarity remain. As shown in Tbl. V, evaluations on image-based VLMs [3], [35] across multiple datasets [22], [42], [75] demonstrate notable speedups

TABLE V
ACCURACY AND SPEEDUP ON IMAGE VLMS

Models	Dataset	Metric	Dense	AdapTiV	Ours
Llava-OneVision	VQAv2	Speedup	1.00	5.19	4.44
		Accuracy	84.32	82.48	83.01
	MME	Speedup	1.00	1.65	4.26
		Score	1067.27	1036.27	1044.79
Qwen2.5-VL	MMBench	Speedup	1.00	1.60	4.25
		Accuracy	84.99	84.49	83.46
	VQAv2	Speedup	1.00	1.96	1.91
		Accuracy	84.48	79.77	81.73
Qwen2.5-VL	MME	Speedup	1.00	1.89	1.97
		Score	1337.66	1129.56	1238.88
	MMBench	Speedup	1.00	1.93	1.78
		Accuracy	85.69	83.79	84.46

over both the systolic-array baseline and AdapTiV, with only minor accuracy degradation. These results indicate that *Focus* effectively removes redundancy beyond the video domain.

Moreover, *Focus* can potentially be extended to Vision–Language–Action (VLA) [30], [33] models for embodied AI applications. VLA models share similar input modalities with VLMS, including image or video inputs paired with text. Therefore, we believe the SEC and SIC in *Focus* could effectively eliminate redundant information in VLA inputs, making this a promising direction for future exploration.

B. Worst- and Best-Case Analysis

Since sparsity varies with video content, we analyze two extreme scenarios to verify robustness. In the **worst case**, when no similarity exists across frames or patches, sparsity drops near zero. The design preserves the full tile length ($m = 1024$), with buffers sized for maximum data without overflow. In the **best case**, abundant similarity yields highly compressed tiles and small m , slightly wasting buffer space and underutilizing the systolic array but remaining correct. We further aggregate the frequency of different tile lengths (i.e. number of vectors) and measure systolic-array utilization (Fig. 13). These extremes are rare, one increases latency, the other lowers utilization, but the system maintains an average utilization of 92.2%, confirming robust performance across diverse inputs.

C. Related Works

Algorithm Optimizations. As a major paradigm for multimodal reasoning, VLMS have attracted significant attention, resulting in a rapidly growing body of work on improving inference efficiency. A large class of recent methods focuses on exploiting redundancy in visual tokens to accelerate VLM inference [7], [20], [29], [43], [44], [54], [62]. FrameFusion [20] compares and merges similar tokens across adjacent frames, while PruneVid [29] performs token clustering and merges tokens within the same cluster. By removing or compressing redundant tokens through diverse algorithmic strategies, these approaches effectively reduce token counts and achieve notable speedups on GPUs.

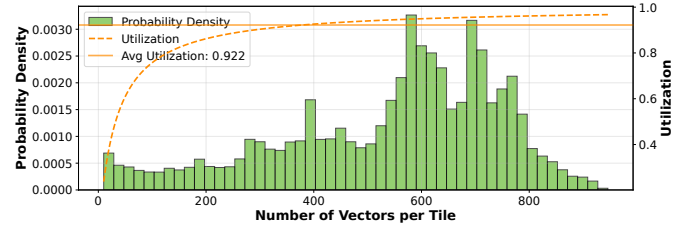


Fig. 13. Histogram and compute utilization of concentrated tile length.

Despite their effectiveness, these methods operate exclusively at the token level and are implemented as software-only optimizations. They are primarily designed for execution on general-purpose GPUs and do not consider how redundancy manifests at finer granularities or how it can be efficiently exploited from a bottom-up hardware perspective.

Architecture Design. From the hardware architecture perspective, to the best of our knowledge, there is no dedicated accelerator specifically designed for VLM inference. Existing architectural efforts instead target efficiency optimizations for LLMs and ViTs, which share the transformer backbone with VLMS. These works predominantly rely on sparsity [16], [23], [60], [61], [63], [64], [71] and quantization [9], [10], [24]–[27]. In terms of sparsity, SpAtten [60] introduces token- and head-level pruning for transformers, while LAD [61] optimizes key–value cache pruning during the decoding phase of LLMs. HeatViT [16] leverages attention maps in ViTs to prune visual tokens. In addition, several works propose hardware-friendly quantization architectures: Olive [24] introduces an outlier–victim pair format integrated into processing element (PE) arrays, and BitMoD [9] enables fine-grained data-type adaptation through bit-serial processing.

While these techniques can be applied to VLMS, they are not explicitly designed for multimodal workloads and therefore may fail to fully capture the unique redundancy patterns introduced by cross-modal interactions. In contrast, *Focus* is the first architecture specifically tailored for VLM inference. By exploiting cross-modal semantic redundancy and detecting fine-grained vector-level similarity, *Focus* enables efficient streaming execution and achieves superior hardware efficiency beyond token-level or modality-agnostic optimizations.

IX. CONCLUSION

We present *Focus*, a streaming concentration architecture that jointly optimizes algorithm and hardware for efficient VLM inference. Our Multilevel Concentration strategy removes redundancy at the semantic, block, and vector levels, while our hardware design performs in-place compression aligned with GEMM tiling and streaming execution. *Focus* achieves up to **2.35×** speedup and **3.29×** energy efficiency improvement over state-of-the-art baselines, with only **2.7%** area overhead in a systolic-array accelerator. By tightly co-designing compression logic with accelerator architecture, *Focus* enables scalable, high-performance deployment of VLMS on both edge and cloud platforms, and paves the way for future hardware-aware multimodal systems.

ACKNOWLEDGMENT

This work was supported in part by NSF-2112562, NSF-2328805, and ARO W911NF-23-2-0224. The authors sincerely thank the anonymous reviewers for their constructive feedback and valuable suggestions that greatly improved the quality of this work. The authors also express their gratitude to Bowen Duan, Jonathan Ku, Yiming Li, and Dr. Tingjun Chen for their technical support and insightful discussions.

APPENDIX

A. Abstract

This artifact provides a complete implementation of *Focus*, a streaming concentration architecture for efficient vision-language model (VLM) inference. The artifact includes three main components: (1) Algorithm implementation of Focus and baseline methods (CMC, Adaptiv, FrameFusion) for multiple VLMs, including LLaVA-Video, LLaVA-OneVision, MiniCPM-V, and Qwen2.5-VL; (2) Cycle-accurate hardware simulator with energy/power estimation and design space exploration capabilities; (3) RTL implementation in Verilog/SystemVerilog. The artifact enables the reproduction of all key results, including accuracy evaluations on the VideoMME, MLVU, MVBench, VQAv2, MME, and MMBench datasets, performance/energy simulations across various design configurations. Generated traces and simulation outputs can be used to reproduce all figures and tables in the evaluation section.

B. Artifact check-list (meta-information)

- **Algorithm:** Focus multilevel concentration (semantic, block, vector levels), CMC, Adaptiv, FrameFusion baselines
- **Program:** Python 3.11+ with PyTorch 2.6.0+
- **Compilation:** Python package installation via pip, third-party code compilation with g++
- **Model:** LLaVA-Video-7B-Qwen2, MiniCPM-V-2.6, LLaVA-OneVision-qwen2-7b-ov, Qwen2.5-VL-7B-Instruct
- **Dataset:** VideoMME, MLVU, MVBench (video); VQAv2, MME, MMBench (image)
- **Run-time environment:** Ubuntu 22.04.2 LTS, CUDA 12.1, PyTorch 2.6.0, Conda environment, HuggingFace Hub access
- **Hardware:** NVIDIA datacenter GPU (A100), multi-core CPU x86_64 processor
- **Execution:** Bash scripts for trace generation, Python scripts for simulation and evaluation, Jupyter notebooks for plotting
- **Metrics:** Model accuracy, sparsity ratio, latency (cycles), energy (mJ), power (W), area (mm²)
- **Output:** CSV files with accuracy/sparsity metrics, PyTorch trace files (.pth), simulation result CSVs, Jupyter notebooks for plotting
- **Experiments:** Trace generation, accuracy evaluation, hardware simulation, design space exploration
- **How much disk space required (approximately)?:** 128GB (models + datasets + traces + codes)
- **How much time is needed to prepare workflow (approximately)?:** 1 hour (installation + model download)
- **How much time is needed to complete experiments (approximately)?:** 6 hours without accuracy evaluation, 480 hours with accuracy evaluation.
- **Publicly available?:** <https://github.com/dubcyfor3/Focus.git>
- **Code licenses (if publicly available)?:** MIT License
- **Data licenses (if publicly available)?:** The datasets are publicly available through their original licensing terms.

- **Workflow automation framework used?:** Bash scripts, Python entry points, Jupyter notebooks
- **Archived (provide DOI)?:** <https://doi.org/10.5281/zenodo.17851346>

C. Description

1) *How to access:* The artifact is available as a Git repository at <https://github.com/dubcyfor3/Focus.git>. Clone the repository and initialize submodules

2) *Hardware dependencies:*

- **GPU:** NVIDIA GPU with 80GB HBM (e.g. A100).
- **CPU:** x86_64 processor
- **Storage:** 128GB+ available disk space

3) *Software dependencies:* The experiments rely on the following software components.

- Ubuntu 22.04+ (tested on Ubuntu 22.04 LTS)
- Python 3.11+
- PyTorch 2.6.0 with CUDA support
- Transformers 4.48.2 (or 4.49.0 for Qwen2.5-VL)
- Accelerate 0.29.1+
- Flash-attention 2.7.4.post1
- g++ compiler
- HuggingFace CLI and account (for model/dataset access)

4) *Data sets:* VideoMME, MLVU, MVBench, VQAv2, MME, MMBench

5) *Models:* The artifact supports multiple pre-trained VLMs accessible via HuggingFace:

- **LLaVA-Video-7B-Qwen2**
(lmms-lab/LLaVA-Video-7B-Qwen2)
- **MiniCPM-V-2.6**
(openbmb/MiniCPM-V-2_6)
- **LLaVA-OneVision**
(lmms-lab/llava-onevision-qwen2-7b-ov)
- **Qwen2.5-VL**
(Qwen/Qwen2.5-VL-7B-Instruct)

D. Installation

We have well-documented README files to detail the installation instructions for each experiment at <https://github.com/dubcyfor3/Focus.git>

E. Experiment workflow

The README file also specifies the detailed experimental workflow for obtaining the results reported in the paper.

F. Evaluation and expected results

Comprehensive README files are provided to document the evaluation procedures for accelerator latency, energy, area, and model accuracy. Expected results can be found in the directories `simulator/example_sim_results` and `algorithm/example_output`.

G. Methodology

Submission, reviewing, and badging methodology:

- <https://www.acm.org/publications/policies/artifact-review-and-badging-current>
- <https://cTuning.org/ae>

REFERENCES

- [1] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, R. Ring, E. Rutherford, S. Cabi, T. Han, Z. Gong, S. Samangooei, M. Monteiro, J. Menick, S. Borgeaud, A. Brock, A. Nematzadeh, S. Sharifzadeh, M. Binkowski, R. Barreira, O. Vinyals, A. Zisserman, and K. Simonyan, "Flamingo: a visual language model for few-shot learning," 2022. [Online]. Available: <https://arxiv.org/abs/2204.14198>
- [2] Amazon Web Services, "Amazon titan multimodal embeddings foundation model," Amazon Bedrock User Guide, 2023, available via Amazon Bedrock: supports embedding of both images and text into a shared semantic space.
- [3] S. Bai, K. Chen, X. Liu, J. Wang, W. Ge, S. Song, K. Dang, P. Wang, S. Wang, J. Tang *et al.*, "Qwen2. 5-vl technical report," *arXiv preprint arXiv:2502.13923*, 2025.
- [4] D. Bolya, C.-Y. Fu, X. Dai, P. Zhang, C. Feichtenhofer, and J. Hoffman, "Token merging: Your vit but faster," in *ICLR*, 2023.
- [5] F. Bordes, R. Y. Pang, A. Ajay, A. C. Li, A. Bardes, S. Petryk, O. Mañas, Z. Lin, A. Mahmoud, B. Jayaraman, M. Ibrahim, M. Hall, Y. Xiong, J. Lebensold, C. Ross, S. Jayakumar, C. Guo, D. Bouchacourt, H. Al-Tahan, K. Padthe, V. Sharma, H. Xu, X. E. Tan, M. Richards, S. Lavoie, P. Astolfi, R. A. Hemmat, J. Chen, K. Tirumala, R. Assouel, M. Moayeri, A. Talattof, K. Chaudhuri, Z. Liu, X. Chen, Q. Garrido, K. Ullrich, A. Agrawal, K. Saenko, A. Celikyilmaz, and V. Chandra, "An introduction to vision-language modeling," *arXiv preprint arXiv:2405.17247*, 2024.
- [6] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [7] L. Chen, H. Zhao, T. Liu, S. Bai, J. Lin, C. Zhou, and B. Chang, "An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models," in *European Conference on Computer Vision*. Springer, 2025, pp. 19–35.
- [8] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," *ACM SIGARCH computer architecture news*, vol. 44, no. 3, pp. 367–379, 2016.
- [9] Y. Chen, A. F. AbouElhamayed, X. Dai, Y. Wang, M. Andronic, G. A. Constantinides, and M. S. Abdelfattah, "Bitmod: Bit-serial mixture-of-datatype llm acceleration," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1082–1097.
- [10] F. Cheng, C. Guo, C. Wei, J. Zhang, C. Zhou, E. Hanson, J. Zhang, X. Liu, H. Li, and Y. Chen, "Ecco: Improving memory bandwidth and capacity for llms via entropy-aware cache compression," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 793–807.
- [11] J. Choquette, W. Gandhi, O. Giroux, N. Stam, and R. Krashinsky, "Nvidia a100 tensor core gpu: Performance and innovation," *IEEE Micro*, vol. 41, no. 2, pp. 29–35, 2021.
- [12] N. Corporation, "Jetson orin nano developer kit," <https://developer.nvidia.com/embedded/learn/get-started-jetson-orin-nano-devkit>, 2023, accessed: 2025-10-17.
- [13] D. Das, D. Talon, M. Mancini, Y. Wang, and E. Ricci, "One VLM to keep it learning: Generation and balancing for data-free continual visual question answering," in *IEEE/CVF Winter Conference on Applications of Computer Vision, WACV 2025, Tucson, AZ, USA, February 26 - March 6, 2025*. IEEE, 2025, pp. 5635–5645. [Online]. Available: <https://doi.org/10.1109/WACV61041.2025.00550>
- [14] T. Dettmers, M. Lewis, S. Shleifer, and L. Zettlemoyer, "8-bit optimizers via block-wise quantization," *arXiv preprint arXiv:2110.02861*, 2021.
- [15] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [16] P. Dong, M. Sun, A. Lu, Y. Xie, K. Liu, Z. Kong, X. Meng, Z. Li, X. Lin, Z. Fang *et al.*, "Heatvit: Hardware-efficient adaptive token pruning for vision transformers," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 442–455.
- [17] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.
- [18] Z. Du, R. Fasthuber, T. Chen, P. Ienne, L. Li, T. Luo, X. Feng, Y. Chen, and O. Temam, "Shidiannao: Shifting vision processing closer to the sensor," in *Proceedings of the 42nd annual international symposium on computer architecture*, 2015, pp. 92–104.
- [19] C. Fu, Y. Dai, Y. Luo, L. Li, S. Ren, R. Zhang, Z. Wang, C. Zhou, Y. Shen, M. Zhang, P. Chen, Y. Li, S. Lin, S. Zhao, K. Li, T. Xu, X. Zheng, E. Chen, C. Shan, R. He, and X. Sun, "Video-mme: The first-ever comprehensive evaluation benchmark of multi-modal llms in video analysis," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 24 108–24 118.
- [20] T. Fu, T. Liu, Q. Han, G. Dai, S. Yan, H. Yang, X. Ning, and Y. Wang, "Framefusion: Combining similarity and importance for video token reduction on large visual language models," *arXiv preprint arXiv:2501.01986*, 2024.
- [21] R. Girdhar, A. El-Nouby, Z. Liu, M. Singh, K. V. Alwala, A. Joulin, and I. Misra, "Imagebind: One embedding space to bind them all," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023, pp. 15 180–15 190.
- [22] Y. Goyal, T. Khot, D. Summers-Stay, D. Batra, and D. Parikh, "Making the v in vqa matter: Elevating the role of image understanding in visual question answering," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 6904–6913.
- [23] C. Guo, B. Y. Hsueh, J. Leng, Y. Qiu, Y. Guan, Z. Wang, X. Jia, X. Li, M. Guo, and Y. Zhu, "Accelerating sparse dnn models without hardware-support via tile-wise sparsity," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2020, pp. 1–15.
- [24] C. Guo, J. Tang, W. Hu, J. Leng, C. Zhang, F. Yang, Y. Liu, M. Guo, and Y. Zhu, "Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–15.
- [25] C. Guo, C. Wei, J. Tang, B. Duan, S. Han, H. Li, and Y. Chen, "Transitive array: An efficient gemm accelerator with result reuse," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 990–1004.
- [26] C. Guo, C. Zhang, J. Leng, Z. Liu, F. Yang, Y. Liu, M. Guo, and Y. Zhu, "Ant: Exploiting adaptive numerical data type for low-bit deep neural network quantization," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 1414–1433.
- [27] W. Hu, H. Zhang, C. Guo, Y. Feng, R. Guan, Z. Hua, Z. Liu, Y. Guan, M. Guo, and J. Leng, "M-ant: Efficient low-bit group quantization for llms via mathematically adaptive numerical type," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1112–1126.
- [28] W. Huang, Z. Zhai, Y. Shen, S. Cao, F. Zhao, X. Xu, Z. Ye, and S. Lin, "Dynamic-llava: Efficient multimodal large language models via dynamic vision-language context sparsification," *arXiv preprint arXiv:2412.00876*, 2024.
- [29] X. Huang, H. Zhou, and K. Han, "Prunevid: Visual token pruning for efficient video large language models," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 19 959–19 973.
- [30] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai *et al.*, "π0. 5: a vision-language-action model with open-world generalization, 2025," URL <https://arxiv.org/abs/2504.16054>, vol. 1, no. 2, p. 3.
- [31] N. P. Jouppi, G. Kurian, S. Li, P. C. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles, C. Young, X. Zhou, Z. Zhou, and D. A. Patterson, "TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings," *CoRR*, vol. abs/2304.01433, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2304.01433>
- [32] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the 44th annual international symposium on computer architecture*, 2017, pp. 1–12.

- [33] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi *et al.*, “Openvla: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [34] H. T. Kung and C. E. Leiserson, “Systolic arrays (for vlsi),” in *Sparse Matrix Proceedings 1978*, vol. 1. Society for industrial and applied mathematics Philadelphia, PA, USA, 1979, pp. 256–282.
- [35] B. Li, Y. Zhang, D. Guo, R. Zhang, F. Li, H. Zhang, K. Zhang, P. Zhang, Y. Li, Z. Liu, and C. Li, “Llava-onevision: Easy visual task transfer,” *arXiv preprint arXiv:2408.03326*, 2024.
- [36] J. Li, D. Li, C. Xiong, and S. C. Hoi, “Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation,” *Proceedings of the International Conference on Machine Learning (ICML)*, 2022.
- [37] K. Li, Y. Wang, Y. He, Y. Li, Y. Wang, Y. Liu, Z. Wang, J. Xu, G. Chen, P. Luo, L. Wang, and Y. Qiao, “Mybench: A comprehensive multi-modal video understanding benchmark,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 22 195–22 206.
- [38] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, “Dramsim3: A cycle-accurate, thermal-capable dram simulator,” *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020.
- [39] C. Lin, J. Tang, S. Yang, H. Wang, T. Tang, B. Tian, I. Stoica, S. Han, and M. Gao, “Twilight: Adaptive attention sparsity with hierarchical top-p pruning,” *arXiv preprint arXiv:2502.02770*, 2025.
- [40] J. Lin, H. Yin, W. Ping, P. Molchanov, M. Shoenybi, and S. Han, “Vila: On pre-training for visual language models,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 26 689–26 699.
- [41] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., 2023. [Online]. Available: http://papers.nips.cc/paper_files/paper/2023/hash/6dcf277ea32ce3288914faf369fe6de0-Abstract-Conference.html
- [42] Y. Liu, H. Duan, Y. Zhang, B. Li, S. Zhang, W. Zhao, Y. Yuan, J. Wang, C. He, Z. Liu *et al.*, “Mmbench: Is your multi-modal model an all-around player?” in *European conference on computer vision*. Springer, 2024, pp. 216–233.
- [43] Y. Liu, J. Sun, Y. Lin, J. Zhang, J. Zhang, M. Yin, Q. Wang, H. Li, and Y. Chen, “Keyframe-oriented vision token pruning: Enhancing efficiency of large vision language models on long-form video processing,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 20 802–20 811.
- [44] Z. Liu, L. Zhu, B. Shi, Z. Zhang, Y. Lou, S. Yang, H. Xi, S. Cao, Y. Gu, D. Li, X. Li, Y. Fang, Y. Chen, C.-Y. Hsieh, D.-A. Huang, A.-C. Cheng, V. Nath, J. Hu, S. Liu, R. Krishna, D. Xu, X. Wang, P. Molchanov, J. Kautz, H. Yin, S. Han, and Y. Lu, “Nvila: Efficient frontier visual language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2412.04468>
- [45] K. Maeda, S. Kurita, T. Miyanishi, and N. Okazaki, “Vision language model-based caption evaluation method leveraging visual context extraction,” *CoRR*, vol. abs/2402.17969, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2402.17969>
- [46] NVIDIA, “Nvidia a100 tensor core gpu architecture,” NVIDIA Corporation, White Paper, 2020.
- [47] OpenAI, J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altschmidt, S. Altman, S. Anadkat, R. Avila, I. Babuschkin, S. Balaji, V. Balcom, P. Baltescu, H. Bao, M. Bavarian, J. Belgum, J. Bello, J. Berdine, G. Bernadett-Shapiro, C. Berner, L. Bogdonoff, O. Boiko, M. Boyd, A.-L. Brakman, G. Brockman, T. Brooks, M. Brundage, K. Button, T. Cai, R. Campbell, A. Cann, B. Carey, C. Carlson, R. Carmichael, B. Chan, C. Chang, F. Chantzi, D. Chen, S. Chen, R. Chen, J. Chen, M. Chen, B. Chess, C. Cho, C. Chu, H. W. Chung, D. Cummings, J. Currier, Y. Dai, C. Decareaux, T. Degry, N. Deutsch, D. Deville, A. Dhar, D. Dohan, S. Dowling, S. Dunning, A. Ecoffet, A. Eleti, T. Eloundou, D. Farhi, L. Fedus, N. Felix, S. P. Fishman, J. Forte, I. Fulford, L. Gao, E. Georges, C. Gibson, V. Goel, T. Gogineni, G. Goh, R. Gontijo-Lopes, J. Gordon, M. Grafstein, S. Gray, R. Greene, J. Gross, S. S. Gu, Y. Guo, C. Hallacy, J. Han, J. Harris, Y. He, M. Heaton, J. Heidecke, C. Hesse, A. Hickey, W. Hickey, P. Hoeschele, B. Houghton, K. Hsu, S. Hu, X. Hu, J. Huizinga, S. Jain, S. Jain, J. Jang, A. Jiang, R. Jiang, H. Jin, D. Jin, S. Jomoto, B. Jonn, H. Jun, T. Kaftan, Łukasz Kaiser, A. Kamali, I. Kanitscheider, N. S. Keskar, T. Khan, L. Kilpatrick, J. W. Kim, C. Kim, Y. Kim, J. H. Kirchner, J. Kiros, M. Knight, D. Kokotajlo, Łukasz Kondraciuk, A. Kondrich, A. Konstantinidis, K. Kosic, G. Krueger, V. Kuo, M. Lampe, I. Lan, T. Lee, J. Leike, J. Leung, D. Levy, C. M. Li, R. Lim, M. Lin, S. Lin, M. Litwin, T. Lopez, R. Lowe, P. Lue, A. Makanju, K. Malfacini, S. Manning, T. Markov, Y. Markovski, B. Martin, K. Mayer, A. Mayne, B. McGrew, S. M. McKinney, C. McLeavey, P. McMillan, J. McNeil, D. Medina, A. Mehta, J. Menick, L. Metz, A. Mishchenko, P. Mishkin, V. Monaco, E. Morikawa, D. Mossing, T. Mu, M. Murati, O. Murk, D. Mely, A. Nair, R. Nakano, R. Nayak, A. Neelakantan, R. Ngo, H. Noh, L. Ouyang, C. O’Keefe, J. Pachocki, A. Paino, J. Palermo, A. Pantuliano, G. Parascandolo, J. Parish, E. Parparita, A. Passos, M. Pavlov, A. Peng, A. Perelman, F. de Avila Belbute Peres, M. Petrov, H. P. de Oliveira Pinto, Michael, Pokorny, M. Pokrass, V. H. Pong, T. Powell, A. Power, B. Power, E. Proehl, R. Puri, A. Radford, J. Rae, A. Ramesh, C. Raymond, F. Real, K. Rimbach, C. Ross, B. Rotsted, H. Roussez, N. Ryder, M. Saltarelli, T. Sanders, S. Santurkar, G. Sastry, H. Schmidt, D. Schnurr, J. Schulman, D. Selsam, K. Sheppard, T. Sherbakov, J. Shieh, S. Shoker, P. Shyam, S. Sidor, E. Sigler, M. Simens, J. Sitkin, K. Slama, I. Sohl, B. Sokolowsky, Y. Song, N. Staudacher, F. P. Such, N. Summers, I. Sutskever, J. Tang, N. Tezak, M. B. Thompson, P. Tillett, A. Tootoonchian, E. Tseng, P. Tuggle, N. Turley, J. Tworek, J. F. C. Uribe, A. Vallone, A. Vijayvergiya, C. Voss, C. Wainwright, J. J. Wang, A. Wang, B. Wang, J. Ward, J. Wei, C. Weinmann, A. Welihinda, P. Welinder, J. Weng, L. Weng, M. Wiethoff, D. Willner, C. Winter, S. Wolrich, H. Wong, L. Workman, S. Wu, J. Wu, M. Wu, K. Xiao, T. Xu, S. Yoo, K. Yu, Q. Yuan, W. Zaremba, R. Zellers, C. Zhang, M. Zhang, S. Zhao, T. Zheng, J. Zhuang, W. Zhuk, and B. Zoph, “Gpt-4 technical report,” 2024. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [48] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “Pytorch: An imperative style, high-performance deep learning library,” *Advances in neural information processing systems*, vol. 32, 2019.
- [49] Y. Rao, W. Zhao, Z. Tang, J. Zhou, and J. Lu, “Dynamicvit: Efficient vision transformers with dynamic token sparsification,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 11 974–11 983.
- [50] S. Sah, R. Kumar, H. Rohmetra, and E. Saboori, “Token pruning using a lightweight background aware vision transformer,” *CoRR*, vol. abs/2410.09324, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2410.09324>
- [51] A. Samajdar, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, “Scale-sim: Systolic cnn accelerator simulator,” *arXiv preprint arXiv:1811.02883*, 2018.
- [52] Y. Shang, M. Cai, B. Xu, Y. J. Lee, and Y. Yan, “Llava-prumerge: Adaptive token reduction for efficient large multimodal models,” *arXiv preprint arXiv:2403.15388*, 2024.
- [53] A. Sharshar, L. U. Khan, W. Ullah, and M. Guizani, “Vision-language models for edge networks: A comprehensive survey,” *IEEE Internet of Things Journal*, 2025.
- [54] L. Shen, G. Gong, T. He, Y. Zhang, P. Liu, S. Zhao, and G. Ding, “Fastvid: Dynamic density pruning for fast video large language models,” *arXiv preprint arXiv:2503.11187*, 2025.
- [55] N. Sinha, V. Jain, and A. Chadha, “Guiding vision-language model selection for visual question-answering across tasks, domains, and knowledge types,” *CoRR*, vol. abs/2409.09269, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2409.09269>
- [56] Z. Song, C. Qi, F. Liu, N. Jing, and X. Liang, “Cmc: Video transformer acceleration via codec assisted matrix condensing,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, pp. 201–215.
- [57] G. Team, P. Georgiev, V. I. Lei, R. Burnell, L. Bai, A. Gulati, G. Tanzer, D. Vincent, Z. Pan, S. Wang, S. Mariooryad, Y. Ding, X. Geng, F. Alcober, R. Frostig, M. Omernick, L. Walker, C. Paduraru, C. Sorokin, A. Tacchetti, C. Gaffney, S. Daruki, O. Sercinoglu, Z. Gleicher, J. Love, P. Voigtlaender, R. Jain, G. Surita, K. Mohamed, R. Blevis, J. Ahn, T. Zhu, K. Kawintiranon, O. Firat, Y. Gu, Y. Zhang, M. Rahtz, M. Faruqui, N. Clay, J. Gilmer, J. Co-Reyes,

- I. Caswell, C. Radebaugh, A. Elisseeff, P. Valenzuela, K. McKinney, K. Paterson, A. Cui, E. Latorre-Chimoto, S. Kim, W. Zeng, K. Durden, P. Ponnappalli, T. Sosea, C. A. Choquette-Choo, J. Manyika, B. Robenek, H. Vashisht, S. Pereira, H. Lam, M. Velic, D. Owusu-Afriyie, K. Lee, T. Bolukbasi, A. Parrish, S. Lu, J. Park, B. Venkatraman, A. Talbert, L. Rosique, Y. Cheng, A. Sozanschi, A. Paszke, P. Kumar, J. Austin, L. Li, K. Salama, B. Perz, W. Kim, N. Dukkupati, A. Baryshnikov, C. Kaplanis, X. Sheng, Y. Chervonyi, C. Unlu, D. de Las Casas, H. Askham, K. Tunyasuvunakool, F. Gimeno, S. Poder, C. Kwak, M. Miecznikowski, V. Mirrokni, A. Dimitriev, A. Parisi, D. Liu, T. Tsai, T. Shevlane, C. Kouridi, D. Garmon, A. Goedeckemeyer, A. R. Brown, A. Vijayakumar, A. Elqursh, S. Jazayeri, J. Huang, S. M. Carthy, J. Hoover, L. Kim, S. Kumar, W. Chen, C. Biles, G. Bingham, E. Rosen, L. Wang, Q. Tan, D. Engel, F. Pongetti, D. de Cesare, D. Hwang, L. Yu, J. Pullman, S. Narayanan, K. Levin, S. Gopal, M. Li, A. Aharoni, T. Trinh, J. Lo, N. Casagrande, R. Vij, L. Matthey, B. Ramadhana, A. Matthews, C. Carey, M. Johnson, K. Goranova, R. Shah, S. Ashraf, K. Dasgupta, R. Larsen, Y. Wang, M. R. Vuyyuru, C. Jiang, J. Ijazi, K. Osawa, C. Smith, R. S. Boppana, T. Bilal, Y. Koizumi, Y. Xu, Y. Altun, N. Shabat, B. Bariach, A. Korchemniy, K. Choo, O. Ronneberger, C. Iwuanyanwu, S. Zhao, D. Soergel, C.-J. Hsieh, I. Cai, S. Iqbal, M. Sundermeyer, Z. Chen, E. Bursztain, C. Malaviya, F. Biadys, P. Shroff, I. Dhillon, T. Latkar, C. Dyer, H. Forbes, M. Nicosia, V. Nikolaev, S. Greene, M. Georgiev, P. Wang, N. Martin, H. Sedghi, J. Zhang, P. Banzal, D. Fritz, V. Rao, X. Wang, J. Zhang, V. Patraucean, D. Du, I. Mordatch, I. Jurin, L. Liu, A. Dubey, A. Mohan, J. Nowakowski, V.-D. Ion, N. Wei, R. Tojo, M. A. Raad, D. A. Hudson, V. Keshava, S. Agrawal, K. Ramirez, Z. Wu, H. Nguyen, J. Liu, M. Sewak, B. Petriani, D. Choi, I. Philips, Z. Wang, I. Bica, A. Garg, J. Wilkiewicz, P. Agrawal, X. Li, D. Guo, E. Xue, N. Shaik, A. Leach, S. M. Khan, J. Wiesinger, S. Jerome, A. Chakladar, A. W. Wang, T. Ornduff, F. Abu, A. Ghaffarkhah, M. Wainwright, M. Cortes, F. Liu, J. Maynez, A. Terzis, P. Samangouei, R. Mansour, T. Kepa, F.-X. Aubet, A. Algyr, D. Banica, A. Weisz, A. Orban, A. Senges, E. Andrejczuk, M. Geller, N. D. Santo, V. Anklin, M. A. Merey, M. Baeuml, T. Strohmman, J. Bai, S. Petrov, Y. Wu, D. Hassabis, K. Kavukcuoglu, J. Dean, and O. Vinyals, "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context," 2024. [Online]. Available: <https://arxiv.org/abs/2403.05530>
- [58] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.
- [59] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [60] H. Wang, Z. Zhang, and S. Han, "Spatten: Efficient sparse attention architecture with cascade token and head pruning," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 97–110.
- [61] H. Wang, Y. Li, H. Xu, Y. Wang, L. Liu, J. Yang, and Y. Han, "Lad: Efficient accelerator for generative inference of llm with locality aware decoding," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1482–1495.
- [62] Q. Wang, H. Ye, M. Chung, Y. Liu, Y. Lin, M. Kuo, M. Ma, J. Zhang, and Y. Chen, "Corematching: A co-adaptive sparse inference framework with token and neuron pruning for comprehensive acceleration of vision-language models," *CoRR*, vol. abs/2505.19235, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2505.19235>
- [63] C. Wei, B. Duan, C. Guo, J. Zhang, Q. Song, H. Li, and Y. Chen, "Phi: Leveraging pattern-based hierarchical sparsity for high-efficiency spiking neural networks," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 930–943.
- [64] C. Wei, C. Guo, F. Cheng, S. Li, H. F. Yang, H. H. Li, and Y. Chen, "Prosperity: Accelerating spiking neural networks via product sparsity," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 806–820.
- [65] T. Wiegand, G. J. Sullivan, G. Bjontegaard, and A. Luthra, "Overview of the h. 264/avc video coding standard," *IEEE Transactions on circuits and systems for video technology*, vol. 13, no. 7, pp. 560–576, 2003.
- [66] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, and A. Rush, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, 2020, pp. 38–45.
- [67] A. Yang, A. Nagrani, P. H. Seo, A. Miech, J. Pont-Tuset, I. Laptev, J. Sivic, and C. Schmid, "Vid2seq: Large-scale pretraining of a visual language model for dense video captioning," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023, pp. 10714–10726.
- [68] Y. Yao, T. Yu, A. Zhang, C. Wang, J. Cui, H. Zhu, T. Cai, H. Li, W. Zhao, Z. He, Q. Chen, H. Zhou, Z. Zou, H. Zhang, S. Hu, Z. Zheng, J. Zhou, J. Cai, X. Han, G. Zeng, D. Li, Z. Liu, and M. Sun, "Minicpm-v: A gpt-4v level mllm on your phone," *arXiv preprint arXiv:2408.01800*, 2024.
- [69] X. Ye, Y. Gan, X. Huang, Y. Ge, Y. Shan, and Y. Tang, "Voco-llama: Towards vision compression with large language models," *arXiv preprint arXiv:2406.12275*, 2024.
- [70] S. Yoo, H. Kim, and J.-Y. Kim, "Adaptiv: Sign-similarity based image-adaptive token merging for vision transformer acceleration," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 64–77.
- [71] H. You, Z. Sun, H. Shi, Z. Yu, Y. Zhao, Y. Zhang, C. Li, B. Li, and Y. Lin, "Vitcod: Vision transformer acceleration via dedicated algorithm and accelerator co-design," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 273–286.
- [72] B. Zhang and R. Sennrich, "Root mean square layer normalization," *Advances in neural information processing systems*, vol. 32, 2019.
- [73] K. Zhang, B. Li, P. Zhang, F. Pu, J. A. Cahyono, K. Hu, S. Liu, Y. Zhang, J. Yang, C. Li, and Z. Liu, "Lmms-eval: Reality check on the evaluation of large multimodal models," 2024. [Online]. Available: <https://arxiv.org/abs/2407.12772>
- [74] Y. Zhang, J. Wu, W. Li, B. Li, Z. Ma, Z. Liu, and C. Li, "Video instruction tuning with synthetic data," *arXiv preprint arXiv:2410.02713*, 2024.
- [75] Y. S. Y. Q. M. Zhang, X. L. J. Y. X. Zheng, K. L. X. S. Y. Wu, R. J. C. Fu, and P. Chen, "Mme: A comprehensive evaluation benchmark for multimodal large language models," *arXiv preprint arXiv:2306.13394*, vol. 18, 2021.
- [76] J. Zhou, Y. Shu, B. Zhao, B. Wu, Z. Liang, S. Xiao, M. Qin, X. Yang, Y. Xiong, B. Zhang, T. Huang, and Z. Liu, "MLvu: Benchmarking multi-task long video understanding," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 13 691–13 701.